Содержание

Введение . . . 4
1 Предпосылки и цели исследования . . . 5
2 Постановка задачи . . . 6
3 Обзор существующих методов . . . 6
3.1 Метод главных компонент . . . 7
3.2 Метод частичных наименьших квадратов . . . 7
3.3 Алгоритм равномерного аппроксимирования и проекции многообразия в постановке с учителем . . . 8
3.4 Метод отбора признаков на основе критерия независимости Гильберта-Шмидта и разреживающей регуляризации . . . 9
4 Предлагаемые методы оценки информативности . . . 9
4.1 Ограничения существующих подходов . . . 9
4.2 Унарная классификация с фоновым классом . . . 10
4.3 Теорема об экстраполяции байесовского классификатора . . . 11
4.4 Состоятельность унарного классификатора . . . 12
4.5 Метод понижения размерности на основе LOFO . . . 13
4.5.1 Идея метода и постановка . . . 13
4.5.2 Унарные модели и зона отказа . . . 13
4.5.3 Пороговое разбиение пространства . . . 14
4.5.4 Подбор порогов по критерию J_k . . . 14
4.5.5 Счётчики попаданий и доли a_1, a_2, b_1, b_2 . . . 14
4.5.6 Унарные метрики F_{12}, G_{12} и агрегированный критерий S . . . 15
4.5.7 Итеративное исключение признаков . . . 15
4.5.8 Вычислительная сложность и ограничения . . . 16
4.6 Метод понижения размерности на основе ансамблей случайных подпространств и SHAP-оценок . . . 16
4.6.1 Ансамбли случайных подпространств (RaSE) . . . 16
4.6.2 Аддитивные объяснения Шепли . . . 17
4.6.3 Описание предлагаемого алгоритма RaSE-SHAP в унарной постановке . . . 17
4.6.4 Преимущества и ограничения метода . . . 19
5 Метрики оценки качества понижения размерности . . . 20
6 Экспериментальное исследование . . . 21
6.1 Синтетический набор данных DS-A . . . 22
6.2 Синтетический набор данных DS-B . . . 24
6.3 Реальный набор данных DS-A-real . . . 26
6.4 Реальный набор данных DS-B-real . . . 28
Заключение . . . 30
Список использованных источников . . . 30

ПРИЛОЖЕНИЯ 32
ПРИЛОЖЕНИЕ А 32
ПРИЛОЖЕНИЕ Б 48
ПРИЛОЖЕНИЕ В 61

Введение

Практика студентов МГТУ им. Н. Э. Баумана является обязательной частью основной образовательной программы высшего образования и представляет собой одну из форм организации учебного процесса.

Практика — это вид учебной работы, направленный на развитие практических навыков и умений, а также на формирование компетенций обучающихся в процессе выполнения определённых видов работ, связанных с будущей профессиональной деятельностью.

Основными видами практики студентов Университета, обучающихся по основным образовательным программам высшего профессионального образования, являются:

– учебная практика;

– производственная практика;

– научно-исследовательская работа (НИР).

Научно-исследовательская работа заключается в освоении студентами средств и приёмов выполнения научно-исследовательских работ, а также в проведении собственно учебно-исследовательской деятельности.

Целями научно-исследовательской работы являются:

– овладение фундаментальной научной базой соответствующего направления подготовки, методологией научного творчества и современными информационными технологиями, а также подготовка к научно-исследовательской деятельности;

– подготовка материалов для выпускной квалификационной работы.

Задачами научно-исследовательской работы являются:

– участие в научно-исследовательском процессе;

– исследование научной темы, выданной студенту;

– обзор источников для выпускной квалификационной работы;

– выполнение теоретической части выпускной квалификационной работы;

– выполнение иных задач, определяемых тематикой и целями исследования.

Расчётно-пояснительная записка по результатам НИР в электронном виде (формат `Word`) после получения зачёта направляется студентом в электронный архив кафедры по адресу: `2archive-fn@mail.ru`.

1 Предпосылки и цели исследования

Домен табличных данных охватывает чрезвычайно широкий круг прикладных задач. К нему относятся банковские и финансовые транзакции, результаты медицинских обследований, показатели промышленного мониторинга, маркетинговые профили пользователей, данные сенсоров кибер-физических систем и многие другие источники структурированной информации. Общей особенностью таких наборов данных является представление объектов в виде строк таблицы, столбцы которой являются признаками, описывающими свойства объектов и их взаимосвязи.

Задачи классификации табличных данных в современных прикладных областях связаны с определением класса объекта по значениям множества признаков. Такие данные часто содержат слабоинформативные или вовсе шумовые признаки. Использование исходного набора признаков без понижения их количества приводит к увеличению вычислительной сложности алгоритмов машинного обучения, решающих задачу классификации, и ухудшению их обобщающей способности. В связи с этим понижение размерности являются ключевыми этапами построения надёжных и эффективных моделей классификации в рамках домена табличных данных.

Понижение размерности направлено на построение компактного представления данных за счёт проекции исходного признакового пространства в пространство меньшей размерности, ориентированное прежде всего на повышение качества и устойчивости моделей классификации. В отличие от этого, оценка информативности признаков является алгоритмом, определяющим вклад каждого исходного признака в процесс принятия решения моделью, и позволяет не только выбирать лучшие признаки, таким образом понижая размерность, но и анализировать их значимость. Такой анализ особенно важен в прикладных областях, например в медицине, где требуется как корректно классифицировать состояние пациента, так и обосновать полученное решение, выявив конкретные показатели, оказывающие наибольшее влияние на постановку диагноза.

Существует большое число методов понижения размерности и оценки информативности признаков табличных данных, которые в общем случае можно разделить на две группы: статистические и нейросетевые. Статистические методы, в большинстве своем, обладают понятной интерпретацией, однако отличаются низкой адаптируемостью к данным со сложными взаимосвязями. Нейросетевые подходы способны моделировать сложные нелинейные зависимости, но требуют значительных объёмов данных, являются вычислительно затратными и нередко слабо обоснованы с точки зрения статистической надёжности и интерпретируемости.

В данной работе рассматриваются новые методы оценки информативности признаков табличных данных в задаче классификации, основанные на использовании унарных моделей [1]. Предлагаются два подхода: метод, использующий идею последовательного исключения признаков по аналогии с LOFO [10], и метод на основе ансамблей случайных подпространств, развивающий идеи RaSE [11]. Оба метода ориентированы на получение устойчивых и интерпретируемых оценок информативности. Эффективность

предложенных методов исследуется на задаче бинарной классификации табличных данных и сравнивается с существующими решениями, как понижения размерности, так и оценки информативности.

2 Постановка задачи

Рассматривается задача классификации табличных данных в бинарной постановке. Пусть задана выборка

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in K \subset \mathbb{R}^d,\ K \text{ - ограниченный компакт},\ y_i \in \{0, 1\}, \tag{1}$$

где каждый объект описывается вектором признаков, включающим как информативные, так и слабоинформативные координаты.

Целью работы является разработка и исследование методов оценки информативности признаков табличных данных, направленных на выявление и отбор наиболее значимых координат признакового пространства для решения задачи бинарной классификации. Под оценкой информативности понимается построение количественных характеристик, отражающих вклад отдельных признаков в различимость классов и качество классификации. В рамках работы предполагается провести сравнение предложенных методов с существующими подходами к понижению размерности и отбору признаков.

Основное внимание уделяется разработке подходов на основе унарной классификации, подробно описанной в работе [1], предполагающих построение отдельных моделей для каждого класса. На основе данной идеи предлагается разработать и исследовать два метода оценки информативности признаков, ориентированных на получение устойчивых и интерпретируемых результатов в улсовиях ограниченного объёма данных.

3 Обзор существующих методов

Поскольку разнообразие методов понижения размерности чрезвычайно велико, в настоящей работе для первичного и репрезентативного сравнения выбраны, с одной стороны, классические статистические подходы к редукции признакового пространства (метод главных компонент, метод частичных наименьших квадратов, а также алгоритм равномерного аппроксимирования и проекций многообразия в постановке с учителем), а с другой — достаточно мощный нелинейный статистический метод оценки информативности признаков на основе критерия независимости Гильберта–Шмидта с разреживающей регуляризацией. Такой выбор позволяет сделать обзор более релевантным и сравнивать предлагаемые методы не только с подходами отбора информативных признаков, но и с методами понижения размерности. Ниже приведено описание, используемых методов.

3.1 Метод главных компонент

Метод главных компонент (англ. Principal Component Analysis) подробно описан в работах [2, 3]. Он строит линейную проекцию данных в пространство меньшей размерности так, чтобы максимально сохранить дисперсию наблюдений. Пусть $X \in \mathbb{R}^{n \times d}$ - матрица центрированных наблюдений. Тогда выборочная ковариационная матрица имеет вид

$$S = \frac{1}{n} X^{\mathsf{T}} X, \tag{2}$$

а первая главная компонента определяется как направление $\mathbf{w}_1 \in \mathbb{R}^d$, максимизирующее дисперсию проекций:

$$\mathbf{w}_1 = \arg \max_{\|\mathbf{w}\|_2 = 1} \mathbf{w}^{\mathsf{T}} S \mathbf{w}. \tag{3}$$

Решение достигается выбором собственного вектора матрицы S, соответствующего наибольшему собственному значению; последующие компоненты $\mathbf{w}_2, \mathbf{w}_3, \ldots$ находятся аналогично с дополнительным условием ортогональности к предыдущим. После выбора $q < d$ компонент матрица проекции $W_q = [\mathbf{w}_1, \ldots, \mathbf{w}_q]$ задаёт пониженную размерность $Z = XW_q$.

Практически метод главных компонент полезен тем, что он прост в реализации, вычислительно эффективен и устраняет мультиколлинеарность признаков за счёт перехода к некоррелированным координатам, что нередко повышает устойчивость последующих моделей. В то же время метод является линейным и не требует обучения: он ориентирован на сохранение общей дисперсии в признаках, а не на различимость классов, поэтому направления, наиболее важные для классификации, могут не совпадать с построенными компонентами. Кроме того, новые координаты являются линейными комбинациями исходных признаков и могут быть трудны для содержательной интерпретации, особенно без дополнительных процедур анализа.

3.2 Метод частичных наименьших квадратов

Метод частичных наименьших квадратов (англ. Partial Least Squares) описан в работе [4]. В отличие от предыдущего метода, этот метод строит латентные переменные, которые одновременно хорошо согласованы с меткой объекта в задаче классификации и при этом объясняют структуру признаков. Пусть $X \in \mathbb{R}^{n \times d}$ - центрированная матрица признаков, а $y \in \mathbb{R}^n$ - центрированный отклик. Тогда первая латентная переменная $t_1 = X\mathbf{w}_1$ выбирается так, чтобы ковариация с откликом была максимальной:

$$\mathbf{w}_1 = \arg \max_{\|\mathbf{w}\|_2 = 1} \operatorname{Cov}(X\mathbf{w},\, y). \tag{4}$$

После нахождения t_1 выполняется дефляция (вычитание из X и y компоненты, объясняемой t_1), и процедура повторяется, формируя последовательность латентных переменных $t_1, t_2, \ldots, t_q$ и соответствующих весов. В результате получается проекция $Z = [t_1, \ldots, t_q]$, ориентированная на сохранение предсказательной информации относитель-

но y.

С практической точки зрения метод частичных наименьших квадратов часто выигрывает у линейных методов, не учитывающих значения меток классов, а также повышает устойчивость к мультиколлинеарности и к режиму $d \gg n$. Ограничением остаётся линейный характер базовой постановки и необходимость выбора числа компонент, обычно по валидационной ошибке; при неверном выборе возможны как потеря полезной информации, так и переобучение. Как и в методе главных компонент, интерпретация латентных переменных требует анализа коэффициентов проекции, поскольку компоненты являются комбинациями исходных признаков.

3.3 Алгоритм равномерного аппроксимирования и проекции многообразия в постановке с учителем

Алгоритм равномерного аппроксимирования и проекции многообразия (англ. Uniform Manifold Approximation and Projection) описан в работах [5, 6]. Он относится к нелинейным методам и строит низкоразмерное вложение, стараясь сохранить локальную структуру соседства: сначала по выборке строится граф ближайших соседей, который интерпретируется как вероятностная структура связности, а затем подбираются точки $z_1, \ldots, z_n \in \mathbb{R}^q$, чтобы структура связности в проекции была согласована с исходной. В базовой формулировке оптимизация сводится к минимизации функции потерь, эквивалентной кросс-энтропии между близостями p_{ij} в исходном пространстве и q_{ij} в проекции:

$$L \;=\; \sum_{i<j} \Big(-\, p_{ij}\, \log q_{ij} \;-\; (1 - p_{ij})\, \log(1 - q_{ij}) \Big), \tag{5}$$

где q_{ij} задаётся сглаженной функцией расстояния между z_i и z_j, например семейством вида $q_{ij} = (1 + a\|z_i - z_j\|_2^{2b})^{-1}$ с параметрами a, b.

В постановке с учителем информация о метках классов используется при построении структуры соседства, усиливая связи внутри классов и ослабляя связи между классами, что приводит к проекциям, более пригодным для анализа разделимости и последующего обучения классификаторов. Достоинством метода является способность отражать нелинейную геометрию данных и сохранять локальные кластеры при понижении размерности, что особенно полезно для визуализации и предварительной обработки. Ограничения связаны с наличием гиперпараметров, стохастичностью оптимизации и слабой интерпретируемостью координат вложения: оси проекции, как правило, не имеют самостоятельного содержательного смысла и не допускают прямой трактовки как комбинаций исходных признаков.

3.4 Метод отбора признаков на основе критерия независимости Гильберта-Шмидта и разреживающей регуляризации

Критерий независимости Гильберта-Шмидта (англ. Hilbert–Schmidt Independence Criterion) подробно рассмотрен в работе [7]. Для выборки $\{(x_i, y_i)\}_{i=1}^n$ строятся матрицы Грама $K \in \mathbb{R}^{n\times n}$ и $L \in \mathbb{R}^{n\times n}$ по ядрам k и ℓ, после чего эмпирическая оценка зависимости выражается через центрирующую матрицу $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\mathsf{T}$:

$$\widehat{\mathrm{HSIC}}(X, Y) = \frac{1}{(n-1)^2}\mathrm{tr}(KHLH). \tag{6}$$

, где $\mathbf{1}$ - вектор из единиц длины n.

Эта форма удобна вычислительно и позволяет учитывать нелинейные зависимости за счёт выбора ядерных функций.

Для высокоразмерного отбора признаков используется выпуклая разреживающая постановка, предложенная, в частности, в работе [8]. Пусть K_j — матрица Грама, построенная только по j-й координате признакового вектора, а $\bar{K}_j = HK_jH$ и $\bar{L} = HLH$ - центрированные версии. Тогда веса $\beta_1, \ldots, \beta_d$ находятся как решение

$$\min_{\beta_1,\ldots,\beta_d \geq 0} \ \frac{1}{2}\left\|\bar{L} - \sum_{j=1}^{d}\beta_j\bar{K}_j\right\|_F^2 \ + \ \lambda\sum_{j=1}^{d}\beta_j, \tag{7}$$

где $\|\cdot\|_F$ - норма Фробениуса, а $\lambda > 0$ управляет разреженностью решения. Ненулевые коэффициенты β_j соответствуют выбранным признакам; такая постановка стремится одновременно к высокой зависимости выбранных признаков с метками классов и к снижению избыточности между признаками, поскольку вклад каждого признака задаётся его собственной матрицей Грама. Сильной стороной подхода является сохранение интерпретируемости за счёт выбора именно исходных признаков и возможность учитывать сложные нелинейные связи через ядра, тогда как ограничения обычно связаны с вычислительными затратами на построение матриц Грама и чувствительностью к выбору ядровых гиперпараметров.

4 Предлагаемые методы оценки информативности

4.1 Ограничения существующих подходов

Так как методы оценки информативности признаков и понижения размерности в задачах классификации табличных данных чрезвычайно разнообразны, их классификация неоднозначна, однако ее можно грубо свести к делению на статистические и нейросетевые подходы и обсудить проблематику каждого из семейств.

Значительная часть статистических подходов опирается на допущения о структуре зависимостей. В ситуациях, когда различимость классов определяется сложными

нелинейными взаимодействиями признаков, такие методы могут давать нестабильные или неполные результаты. В свою очередь, нейросетевые подходы действительно способны моделировать сложные нелинейные зависимости, но при этом возникает принципиальная трудность, связанная с корректностью вывода вне области, поддерживаемой данными. В строгой вероятностной постановке условное математическое ожидание $g(x) = \mathbb{E}(Y \mid X = x)$ определено единственным образом лишь почти наверное; вне носителя S распределения P_X значения $g(x)$ не фиксируются данными и могут быть произвольными. Следовательно, любая модель, которая неявно продолжает правило принятия решений на весь компакт признакового пространства, вынуждена осуществлять экстраполяцию в области $K \setminus S$, где отсутствует статистическое обоснование. Это делает оценки информативности, полученные на всем компакте, потенциально неустойчивыми: даже небольшие изменения выборки, появление выбросов или сдвиг распределения, могут резко изменить поведение модели вне носителя и тем самым исказить итоговые объяснения.

Решение данной проблемы изучается в области доверенного искусственного интеллекта и требуется в прикладных областях, где критически важно не только получить ранжирование признаков, но и быть уверенным, что оно сформировано там, где вывод подкреплён данными, например, в медицине и инженерии. В данной работе вводится принципиальное требование: алгоритм должен иметь механизм отказа от принятия решения, то есть формально выделять область признакового пространства, где надёжное решение и интерпретация невозможны.

В качестве базового инструмента для реализации зоны отказа и получения статистически обоснованных оценок далее используется идея унарной классификации с добавлением равномерного распределения фона, предложенная в статье [1]. Такой переход позволяет построить модель, которая обучается на наблюдаемом носителе данных и одновременно закрепляет корректное поведение вне носителя через явную унарную постановку, что делает последующие процедуры оценки информативности более устойчивыми и интерпретируемыми.

4.2 Унарная классификация с фоновым классом

Рассмотрим компакт $K = [0, 1]^d$. В рамках унарной постановки классификации, для каждого класса строится собственный независиый классификатор: фиксируется целевой класс, а обучающая выборка дополняется фоновыми объектами, равномерно распределёнными на K и имеющими нулевую метку. Такая постановка прямо отражает практическую идею: модель должна уметь отличать объекты целевого класса от любых других, то есть заведомо не принадлежащих наблюдаемому носителю целевого распределения. На практике берутся n наблюдений целевого класса с меткой $+1$ и добавляются независимые фоновые наблюдения, равномерно распределённые на K, с нулевой меткой. Таким образом строится по одной модели на каждый класс, а решение о принадлежности объекта к какому-либо из классов определяется на основе выходов

всех унарных классиификаторов.

Ключевым объектом в такой схеме является регрессионная функция на смеси данных класса с фоном. В [1] предлагается рассматривать модификацию исходного распределения P в виде смеси

$$P_\alpha = (1-\alpha)P + \alpha\widetilde{P}, \tag{8}$$

где $\widetilde{P}$ соответствует равномерно распределённому на $[0,1]^d$ вектору признаков с нулевой меткой. И исследуется байесовская регрессия на модифицированном распределении

$$g_\alpha(x) = \mathbb{E}_{P_\alpha}(Y \mid X = x). \tag{9}$$

Такая модификация позволяет зафиксировать поведение оптимального правила в местах не подкрепленных данными: в унарной постановке $g_\alpha(x)$ совпадает с условной вероятностью целевого класса, а вне носителя данных обращается в нуль, формируя естественную зону отказа.

4.3 Теорема об экстраполяции байесовского классификатора

В работе [1] получено строгое описание функции g_α и тем самым - оптимального экстраполированного правила классификации. В частности, показывается, что решение задачи регрессии на смеси существует, почти всюду единственно и допускает явную запись через разложение меры P_X относительно меры Лебега. А именно, существует неотрицательная интегрируемая плотность ρ и борелевское множество $A \subseteq S$ нулевой меры Лебега, такие что для почти всех $x \in [0,1]^d$ выполнено

$$g_\alpha(x) = \begin{cases} g(x), & x \in A, \\ \dfrac{(1-\alpha)\, g(x)\, \rho(x)}{\alpha + (1-\alpha)\rho(x)}, & \rho(x) > 0,\ x \in S \setminus A, \\ 0, & \rho(x) = 0,\ x \in S \setminus A \ \text{ или } \ x \notin S, \end{cases} \tag{10}$$

где $g(x) = \mathbb{E}(Y \mid X = x)$. Отсюда следует важный практический вывод: область $\{x : g_\alpha(x) = 0\}$ покрывает $[0,1]^d \setminus S$ почти всюду и тем самым реализует механизм отказа от решения там, где отсутствуют данные и где стандартное обучение вынуждено делать произвольную экстраполяцию.

Для задач оценки информативности признаков это принципиально: если модель способна отделять область обоснованного вывода от области отказа, то важность признаков можно определять и анализировать на доверенной области, снижая влияние выбросов, сдвигов распределения и прочих эффектов.

4.4 Состоятельность унарного классификатора

Чтобы использовать унарную классификацию как основу алгоритмов информативности, для реализации доверенного метода понижения размерности, способного улавливать сложные зависимости, необходимо понимать, насколько оценка g_α статистически надёжна и как её интерпретировать. В работе [9] доказывается, что выход многослойного персептрона с кусочно-линейными функциями активации при обучении по среднеквадратичной ошибке в унарной постановке может рассматриваться как состоятельная оценка гистограммной регрессии, построенной по ячейкам линейности, индуцированным многослойным персептроном. Данный результат получен, как на практике, так и в виде следующей теоремы:

Теорема 1 (о сходимости нейросетевой и гистограммной регрессий) [9]. Пусть заданы компакт K, $D = \operatorname{diam}(K)$ и последовательность многослойных персептронов с модульной функцией активации, обученных на выборке из n наблюдений из распределения с ограниченной плотностью на K, архитектура которых состоит из первого слоя ширины r_n, L_n слоёв ширины k_n и одноэлементного последнего слоя с линейной активацией. Предположим, что весовые коэффициенты инициализируются независимо из непрерывного распределения, а параметры первого слоя заморожены при обучении. Предположим также, что целевая функция является кусочно гладкой, $g \in \mathcal{F}_{\mathrm{MF},J,\alpha,\beta}$, и выполнены следующие ограничения:

(i) Число ненулевых параметров:

$$S_{\mathrm{nnz},n} = c_1' \max\Big\{ n^{\frac{d}{2\beta+d}},\ n^{\frac{d-1}{\alpha+d-1}} \Big\}. \tag{11}$$

(ii) Ограничение на величины параметров:

$$B_n \le c_1\, n^s. \tag{12}$$

(iii) Количество слоёв после первого:

$$L_n \le c_1 \Big(1 + \max\Big\{ \frac{\beta}{d},\ \frac{\alpha}{2(d-1)} \Big\} \Big). \tag{13}$$

(iv) Число нейронов в первом слое:

$$r_n > 2d. \tag{14}$$

(v) Ограничение на скорость роста архитектуры:

$$k_n^{L_n \min\{d,k_n\}}\, r_n^d = o(n), \qquad \frac{k_n^{L_n \min\{d,k_n\}}\, r_n^d\, d(k_n L_n + r_n) \log(k_n L_n + r_n)\, \log n}{n} \to 0. \tag{15}$$

(vi) Ограничения на ширину первого слоя (для произвольного $\gamma > 0$):

$$\sum_{n=1}^{\infty} \exp(-c\gamma^4 r_n) < \infty, \qquad r_n > C\gamma^{-12}\frac{\omega(K)^2}{D^{10}}. \tag{16}$$

Тогда

$$c_n(X) - h_n(X) \xrightarrow{P} 0, \qquad n \to \infty. \tag{17}$$

Привеленные теоремы из работ по унарной классификации дают два ключевых вывода, которые становятся фундаментом для построения доверенных оценок информативности признаков. Во-первых, унарная нейросетевая модель, обученная на смеси распределения целевого класса и распределния равномерного фона, имеет строгую статистическую опору: её выход можно рассматривать как корректную асимптотическую оценку байесовской регрессии g_α, а значит, она наследует принципиальное свойство унарной постановки - формирование естественной зоны отказа вне носителя данных. Во-вторых, тот же результат объясняет интерпретируемость: нейросетевая регрессия асимптотически эквивалентна гистограммной регрессии h_n на ячейках линейности. В совокупности это означает, что унарная классификация одновременно сочетает нелинейную выразительность с математически обоснованным поведением в областях, где нет данных, и интерпретируемой структурой оценки, что делает её удобной базой для конструирования устойчивых методов оценки информативности признаков.

Далее будут предложены две новые идеи оценки информативности признаков табличных данных с использованием унарных классификаторов.

4.5 Метод понижения размерности на основе LOFO

4.5.1 Идея метода и постановка

В качестве первого предлагаемого метода рассматривается подход оценки информативности признаков, построенный по схеме последовательного исключения признаков по одному с переобучением моделей после исключения каждого признака (Leave-One-Feature-Out). Схема LOFO подробно описана в статье [10] и в данной работе адаптирована к классификации в унарной постоновке. Ключевая цель метода сохранить подмножество признаков фиксированного размера $q < d$, которое обеспечивает максимальную различимость классов в смысле унарных метрик, определённых ниже.

4.5.2 Унарные модели и зона отказа

Пусть в бинарной задаче выделены два класса ω_1 и ω_2, и для каждого класса обучается отдельная унарная модель, отличающая объекты данного класса от фона. Обозначим через $c_1(x)$ и $c_2(x)$ выходы двух унарных классификаторов, построенных для классов ω_1 и ω_2 соответственно. В соответствии с предыдущими разделами, данные каждого класса дополняются фоновыми объектами, равномерно распределёнными на

компакте K, что формирует статистически обоснованную зону отказа: вне носителя данных унарные выходы стремятся к нулю, а решение становится консервативным.

4.5.3 Пороговое разбиение пространства

Для перехода от вещественных выходов $c_1(x), c_2(x)$ к определению класса вводятся пороги $\beta_1, \beta_2 \in \mathbb{R}$. На их основе пространство признаков разбивается на четыре области:

$$R_1(\beta_1, \beta_2) = \{x : \ c_1(x) \geq \beta_1, \ c_2(x) < \beta_2\}, \tag{18}$$

$$R_2(\beta_1, \beta_2) = \{x : \ c_2(x) \geq \beta_2, \ c_1(x) < \beta_1\}, \tag{19}$$

$$R_{12}(\beta_1, \beta_2) = \{x : \ c_1(x) \geq \beta_1, \ c_2(x) \geq \beta_2\}, \tag{20}$$

$$R_0(\beta_1, \beta_2) = \{x : \ c_1(x) < \beta_1, \ c_2(x) < \beta_2\}. \tag{21}$$

Здесь R_1 и R_2 соответствуют однозначному принятию решения в пользу одного из классов, R_{12} описывает область наложения, а R_0 соответствует зоне отказа, что является принципиальным отличием от стандартной бинарной схемы без механизма отказа.

4.5.4 Подбор порогов по критерию J_k

Пороги β_1, β_2 должны удовлетворять двум требованиям: высокая доля корректно распознанных объектов целевых классов и низкая доля ложных срабатываний на фоне, а также на противоположном классе. Для каждого класса $k \in \{1, 2\}$ можно рассматривать критерий в виде:

$$J_k(\beta_k) = \mathrm{TPR}_k(\beta_k) + \mathrm{TNR}_k(\beta_k) - 1, \tag{22}$$

где $\mathrm{TPR}_k(\beta_k)$ есть доля объектов класса ω_k с $c_k(x) \geq \beta_k$, а $\mathrm{TNR}_k(\beta_k)$ есть доля объектов фона и, объектов второго класса, для которых выполняется $c_k(x) < \beta_k$. Максимизация $J_k(\beta_k)$ по β_k даёт порог, который балансирует чувствительность и специфичность унарного классификатора в присутствии фона.

4.5.5 Счётчики попаданий и доли a_1, a_2, b_1, b_2

Пусть на валидационной выборке заданы подвыборки классов $\mathcal{D}_{\mathrm{val}}^{(1)} = \{x_i : \ y_i = \omega_1\}$ и $\mathcal{D}_{\mathrm{val}}^{(2)} = \{x_i : \ y_i = \omega_2\}$ размеров n_1 и n_2. Введём счётчики попаданий объектов

каждого класса в области R_1, R_2, R_{12}:

$$n_{10}^{(1)} = \sum_{i:\, y_i=\omega_1} \mathbf{1}\{x_i \in R_1(\beta_1, \beta_2)\}, \tag{23}$$

$$n_{02}^{(2)} = \sum_{i:\, y_i=\omega_2} \mathbf{1}\{x_i \in R_2(\beta_1, \beta_2)\}, \tag{24}$$

$$n_{12}^{(1)} = \sum_{i:\, y_i=\omega_1} \mathbf{1}\{x_i \in R_{12}(\beta_1, \beta_2)\}, \tag{25}$$

$$n_{12}^{(2)} = \sum_{i:\, y_i=\omega_2} \mathbf{1}\{x_i \in R_{12}(\beta_1, \beta_2)\}. \tag{26}$$

Далее определим доли однозначного распознавания и доли наложения:

$$a_1 = \frac{n_{10}^{(1)}}{n_1}, \qquad a_2 = \frac{n_{02}^{(2)}}{n_2}, \qquad b_1 = \frac{n_{12}^{(1)}}{n_1}, \qquad b_2 = \frac{n_{12}^{(2)}}{n_2}. \tag{27}$$

Величины a_1, a_2 измеряют, какая доля объектов каждого класса попадает в область однозначного решения, то есть характеризуют эффективность разделения с учётом зоны отказа. Величины b_1, b_2 измеряют долю объектов каждого класса, попадающих в область наложения, то есть характеризуют неоднозначность решения.

4.5.6 Унарные метрики F_{12}, G_{12} и агрегированный критерий S

Для получения симметричных по классам скалярных характеристик используются гармонические средние:

$$F_{12} = \frac{2a_1a_2}{a_1 + a_2 + \varepsilon}, \qquad G_{12} = \frac{2b_1b_2}{b_1 + b_2 + \varepsilon}, \tag{28}$$

где $\varepsilon > 0$ - малый параметр, предотвращающий деление на нуль. Метрика F_{12} является аналогом $F1$-меры, отражая согласованно высокую эффективность по обоим классам, тогда как G_{12} отражает степень наложения классов, которая нежелательна. Для окончательной оценки качества вводится агрегированный критерий

$$S(\beta_1, \beta_2; c_G) = F_{12} - c_G\, G_{12}, \qquad c_G > 0, \tag{29}$$

где параметр c_G задаёт штраф за наложение. Большие значения S соответствуют ситуациям, когда классы разделяются эффективно и при этом редко попадают в область конфликта R_{12}.

4.5.7 Итеративное исключение признаков

Пусть $J \subseteq \{1, \ldots, d\}$ - текущий набор сохраняемых признаков. Для каждого кандидата $j \in J$ строится редуцированная выборка, в которой удалена координата j, то есть рассматривается проекция $x_{-j} \in \mathbb{R}^{|J|-1}$. На этой редуцированной выборке переобучаются унарные модели $c_{1,-j}$ и $c_{2,-j}$, выбираются пороги $\beta_{1,-j}, \beta_{2,-j}$ и вычисляется

значение критерия S_{-j} на валидации. После этого исключается признак

$$j^\star = \arg\max_{j \in J} S_{-j}, \tag{30}$$

то есть удаляется координата, исключение которой приводит к наибольшему улучшению или наименьшему ухудшению разделимости в смысле унарных метрик. Процедура повторяется итеративно до тех пор, пока не останется q признаков. Итогом работы метода является подмножество признаков J, которое можно использовать как результат понижения размерности.

Код реализации предлагаемого метода представлен в приложении А.

4.5.8 Вычислительная сложность и ограничения

Схема LOFO требует переобучения моделей при каждом исключении признака. В простейшей реализации вычислительная сложность растёт как $O(d(d-q))$ обучений унарных моделей, что ограничивает применение при больших d.

Дополнительным ограничением является проклятие размерности, связанное с использованием равномерного фонового распределения в процессе обучения моделей. В высоких размерностях объём гиперкуба быстро растёт, и количество данных фона, необходимых для полноценного покрытия всего гиперкуба, растет экспоненциально.

4.6 Метод понижения размерности на основе ансамблей случайных подпространств и SHAP-оценок

В качестве второго предлагаемого метода рассматривается подход, сочетающий идею ансамблей случайных подпространств и унарную классификацию с зоной отказа, а также использование аддитивных объяснений Шепли для обновления оценок информативности признаков. Цель метода - получить устойчивый вектор информативности исходных координат и затем выбрать $q < d$ наиболее значимых признаков, при этом поборов проклятие размерности, наследуемое от обучения унарных классификаторов.

4.6.1 Ансамбли случайных подпространств (RaSE)

Метод ансамблей случайных подпространств (англ. Random Subspace Ensemble), известный как RaSE, предлагается в работе [11]. Его основная идея состоит в том, что вместо обучения одной сложной модели на всём пространстве $\mathbb{R}^d$ строится ансамбль слабых моделей, каждая из которых обучается на подпространстве признаков небольшой размерности. Подпространства выбираются случайно, но не равномерно: вероятность включения координат адаптивно обновляется так, чтобы чаще выбирать признаки, встречающиеся в наиболее качественных подпространствах. В [11] приведён детальный теоретический анализ и обоснование такой схемы.

Достоинство данной идеи в контексте унарной классификации заключается в следующем: при фиксированном размере подпространства $m \ll d$ обучение и валидация унарных моделей становятся существенно устойчивее и дешевле, а генерация фоновых объектов равномерного шума в m-мерном гиперркубе требует существенно меньшего числа точек, чем в d-мерном.

4.6.2 Аддитивные объяснения Шепли

Для измерения вклада координат внутри каждого выбранного подпространства используются SHAP-оценки (англ. SHapley Additive explanations), предложенные в работе [12]. SHAP рассматривает объяснение предсказания как аддитивную модель по признакам:

$$f(x) \;=\; \phi_0 \;+\; \sum_{j=1}^{d} \phi_j(x), \tag{31}$$

где $f(x)$ - выход модели в точке x, ϕ_0 - базовый уровень, а $\phi_j(x)$ - вклад j-й координаты.

Вклад признака $\phi_j(x)$ определяется как значение Шепли из теории игр:

$$\phi_j(x) \;=\; \sum_{S \subseteq \{1,\dots,d\} \setminus \{j\}} \frac{|S|!\,(d-|S|-1)!}{d!} \Big(f_{S \cup \{j\}}(x_{S \cup \{j\}}) - f_S(x_S) \Big), \tag{32}$$

где S - подмножество признаков, не содержащее j, $f_S(x_S)$ - значение модели, вычисленное при использовании только признаков из S, а остальные координаты считаются отсутствующими.

В [12] показано, что в классе аддитивных объяснений, удовлетворяющих аксиомам локальной точности, симметрии и отсутствия вклада для неиспользуемых признаков, существует единственное решение, совпадающее со значениями Шепли, что делает SHAP принципиально обоснованной мерой важности признаков.

4.6.3 Описание предлагаемого алгоритма RaSE-SHAP в унарной постановке

Пусть решается бинарная задача с классами ω_1, ω_2. Как и ранее, обучаются два унарных классификатора: $c_1(x)$ для класса ω_1 и $c_2(x)$ для класса ω_2, причём обучение каждого унарного классификатора проводится на множестве целевых объектов, смешанных с фоновыми объектами, равномерно распределёнными на компакте в соответствующем подпространстве признаков.

Метод конструирует вектор вероятностей выбора координат

$$\pi = (\pi_1, \dots, \pi_d), \qquad \pi_j \geq 0, \quad \sum_{j=1}^{d} \pi_j = 1, \tag{33}$$

который интерпретируется как текущая оценка информативности признаков. Итерации обновления веектора вероятностей состоят из трёх этапов.

Генерация и отбор подпространств. Фиксируется размер подпространства m и число кандидатов. На каждой итерации генерируются подпространства-кандидаты $J \subset \{1, \ldots, d\}$, $|J| = m$: с вероятностью λ координаты выбираются равномерно, а с вероятностью $1 - \lambda$ - согласно распределению π, что обеспечивает возможность преодолеть проблемы зависания метода. Для каждого кандидата J обучаются унарные модели $c_{1,J}$ и $c_{2,J}$ на признаках x_J и на валидации подбираются пороги $\beta_{1,J}, \beta_{2,J}$, максимизирующие критерий разделимости

$$S_J = F_{12}(\beta_{1,J}, \beta_{2,J}) - c_G\, G_{12}(\beta_{1,J}, \beta_{2,J}), \tag{34}$$

где F_{12} и G_{12} определены согласно предыдущему методу, а $c_G > 0$ задаёт штраф за область наложения. Далее сохраняется набор лучших подпространств и соответствующих моделей по величине S_J.

Оценка важности координат внутри лучших подпространств через SHAP. Для каждого выбранного лучшего подпространства J вычисляются SHAP-оценки для унарных моделей. Обозначим через $\phi_J^{(1)}(x) \in \mathbb{R}^m$ вектор SHAP-вкладов для модели $c_{1,J}$, а через $\phi_J^{(2)}(x)$ - для модели $c_{2,J}$. Для устойчивости далее используются только неотрицательные вклады, так как только они влияют положительно на принадлежность данных к конкретному классу, и усредним результат по объектам соответствующего класса на валидации:

$$u_J^{(1)} = \frac{1}{|V_1|} \sum_{x \in V_1} \max(\phi_J^{(1)}(x), 0), \qquad u_J^{(2)} = \frac{1}{|V_2|} \sum_{x \in V_2} \max(\phi_J^{(2)}(x), 0), \tag{35}$$

где V_1 и V_2 - валидационные подвыборки классов ω_1 и ω_2. Затем векторы нормируются на сумму и объединяются в симметричную оценку значимости координат подпространства через гармоническое среднее:

$$v_J = \frac{2\, u_J^{(1)} \odot u_J^{(2)}}{u_J^{(1)} + u_J^{(2)} + \varepsilon}, \tag{36}$$

где $\odot$ - поэлементное умножение, $\varepsilon > 0$ - малый параметр. Такое объединение усиливает признаки, которые одновременно важны для обоих унарных классификаторов, и подавляет признаки, значимые лишь для одного из них.

Обновление распределения π. Пусть $\mathcal{J}_{\text{top}}$ - набор лучших подпространств на текущей итерации. Из векторов v_J формируется общий список информативности по всем признакам согласно отобранным кандидатам подпространств.

$$w_i = \sum_{J \in \mathcal{J}_{\text{top}}} \mathbf{1}\{i \in J\}\, (v_J)_{\text{pos}(i,J)}, \tag{37}$$

где $\text{pos}(i, J)$ - позиция признака i внутри упорядоченного списка индексов J. После нормировки w по полученным результатам с использованием сглаживания обновляется

общий вектор информативности признаков

$$\pi^{\text{new}} = (1-\alpha)\pi + \alpha \frac{w}{\sum_{j=1}^{d} w_j}, \qquad \alpha \in (0,1), \tag{38}$$

а затем выполняется нормировка с ограничениями снизу и сверху $L \le \pi_j \le U$, предотвращающая вырождение распределения.

$$\pi \leftarrow \arg\min_{p} \|p - \pi^{\text{new}}\|_2^2 \quad \text{при} \quad \sum_{j=1}^{d} p_j = 1, \;\; L \le p_j \le U. \tag{39}$$

Наличие нижней границы L обеспечивает неизчезающую вероятность исследования координат, а верхняя граница U не позволяет π слишком рано схлопнуться в несколько признаков.

После выполнения заданного числа итераций итоговая оценка информативности задаётся вектором π. Как результат понижения размерности выбирается множество индексов q наибольших компонентов π. Дополнительно помимо вектора информативности признаков в качестве результата работы метода получен ансамбль унарных классификаторов, который решает задачу классификации в исходном пространстве признаков.

Код реализации данного метода находится в приложении Б.

4.6.4 Преимущества и ограничения метода

Ключевым преимуществом предлагаемого метода является частичное преодоление проклятия размерности. В отличие от прямого обучения унарных моделей на всём пространстве признаков, обучение и валидация выполняются на подпространствах существенно меньшей размерности, что снижает требования к числу наблюдений и к объёму фоновых данных, необходимых для формирования корректной зоны отказа. Адаптивный характер выбора подпространств позволяет концентрировать вычислительные ресурсы на наиболее информативных координатах и тем самым получать более устойчивые оценки информативности в условиях высокой размерности и ограниченного объёма выборки.

В то же время метод остаётся вычислительно сложным. На каждой итерации требуется обучение нескольких унарных моделей на различных подпространствах, подбор порогов и вычисление SHAP-оценок, что приводит к значительным затратам времени и памяти, особенно при большом числе признаков и при использовании нейросетевых базовых моделей. Таким образом, практическое применение метода требует компромисса между точностью оценок информативности и доступными вычислительными ресурсами.

Дополнительным ограничением является необходимость задания набора гиперпараметров и констант, включая размер подпространств, число моделей в ансамбле, параметры сглаживания распределения вероятностей координат и границы допустимых значений этих вероятностей. Некорректный выбор данных констант может при-

водить как к излишне медленной сходимости алгоритма, так и к преждевременному вырождению распределения информативности. Вопрос о теоретически обоснованном и практически устойчивом выборе таких параметров в различных режимах работы алгоритма требует отдельного исследования и будет рассматриваться в дальнейшем на основе более глубоких теоретических рассуждений и анализа свойств предлагаемого метода.

5 Метрики оценки качества понижения размерности

Оценка качества понижения размерности в данной работе проводится по тому, как редуцированное пространство влияет на обучение и качество произвольных моделей классификации. Такой подход является принципиальным: цель понижения размерности в задачах классификации заключается не в сохранении структуры данных как таковой, а в сохранении или улучшении предсказательной способности моделей, обучаемых на уменьшенном наборе признаков.

В экспериментальном исследовании для этой цели используются две модели: многослойный персептрон и градиентный бустинг над гистограммной регрессией. Для каждой модели проводится обучение на исходном пространстве признаков и на пространстве, полученном после применения метода понижения размерности до фиксированной размерности q. Качество сравнивается по стандартным метрикам бинарной классификации, включая площадь под ROC-кривой (ROC AUC) и площадь под PR-кривой (PR AUC).

Для вывода модели $s(x) \in \mathbb{R}$, по которому строится бинарное решение $\widehat{y}_\tau = \mathbf{1}\{s(x) \geq \tau\}$ при пороге τ, обозначим через $\mathrm{TP}(\tau), \mathrm{FP}(\tau), \mathrm{TN}(\tau), \mathrm{FN}(\tau)$ числа истинно положительных, ложно положительных, истинно отрицательных и ложно отрицательных предсказаний при пороге τ. Тогда вводятся

$$\mathrm{TPR}(\tau) = \frac{\mathrm{TP}(\tau)}{\mathrm{TP}(\tau) + \mathrm{FN}(\tau)}, \qquad \mathrm{FPR}(\tau) = \frac{\mathrm{FP}(\tau)}{\mathrm{FP}(\tau) + \mathrm{TN}(\tau)}. \tag{40}$$

ROC-кривая - это зависимость $\mathrm{TPR}(\tau)$ от $\mathrm{FPR}(\tau)$ при изменении τ, а метрика ROC AUC определяется как площадь под этой кривой:

$$\text{ROC AUC} = \int_0^1 \mathrm{TPR}(u)\, du, \qquad u = \mathrm{FPR}(\tau). \tag{41}$$

Эквивалентная вероятностная интерпретация состоит в том, что ROC AUC равна вероятности того, что случайно выбранный объект положительного класса получит больший вывод модели, чем вывод случайно выбранного объекта отрицательного класса:

$$\text{ROC AUC} = \mathbb{P}\big(s(X^+) > s(X^-)\big) + \frac{1}{2}\mathbb{P}\big(s(X^+) = s(X^-)\big). \tag{42}$$

Для PR-кривой используются точность (precision) и полнота (recall). При пороге

τ:

$$\text{Precision}(\tau) = \frac{\text{TP}(\tau)}{\text{TP}(\tau) + \text{FP}(\tau)}, \qquad \text{Recall}(\tau) = \text{TPR}(\tau) = \frac{\text{TP}(\tau)}{\text{TP}(\tau) + \text{FN}(\tau)}. \tag{43}$$

PR-кривая - это зависимость Precision(τ) от Recall(τ) при изменении τ, а метрика PR AUC (средняя точность) определяется как площадь под PR-кривой:

$$\text{PR AUC} = \int_0^1 \text{Precision}(r)\, dr, \qquad r = \text{Recall}(\tau). \tag{44}$$

Данная метрика особенно чувствительна к качеству ранжирования в области малых ложноположительных срабатываний и обычно более информативна при дисбалансе классов.

Дополнительно анализируются разности

$$\Delta\text{AUC} = \text{AUC}_{\text{red}} - \text{AUC}_{\text{base}}, \qquad \Delta\text{PR} = \text{PR}_{\text{red}} - \text{PR}_{\text{base}},$$

которые напрямую отражают выигрыш или потерю качества вследствие понижения размерности для фиксированной модели.

Помимо предсказательных характеристик, учитываются вычислительные затраты. Измеряется время работы алгоритма понижения размерности t_{reducer}, а также время обучения модели на редуцированном и на исходном пространствах признаков t_{model} и $t_{\text{model,base}}$ соответственно. Такой набор метрик позволяет оценивать методы понижения размерности комплексно, с учётом компромисса между качеством классификации и вычислительной эффективностью, что особенно важно в прикладных сценариях с ограниченными вычислительными ресурсами.

6 Экспериментальное исследование

В экспериментальном исследовании сравниваются два предложенных метода - унарный LOFO (MLP_unar) и ансамбль случайных подпространств с SHAP-оценками (Ensembleunar) - с базовыми методами из обзора: метод главных компонент (PCA), метод частичных наименьших квадратов (PLS), алгоритм Uniform Manifold Approximation and Projection в постановке с учителем (UMAP_sup), а также метод отбора признаков на основе критерия независимости Гильберта-Шмидта и разреживающей регуляризации (HSIC_Lasso). Отметим, что в сравнении присутствуют как методы понижения размерности (PCA, PLS, UMAP_sup), так и методы оценки информативности (HSIC_Lasso, MLP_unar, Ensembleunar), по этой причине методы оценки информативности, только отбирающие существующие признаки, не всегда могут лидировать. однако в данной работе рассматриваются оба семейства методов для более полного анализа. В дальнейшем планируется расширить сравнение за счёт большего числа альтернативных методов и стратегий настройки.

Способы оценки качества работы модели, используемые в экспериментах, описаны ранее.

Код реализации всех экспериментов находится в приложении B.

6.1 Синтетический набор данных DS-A

Набор данных DS-A представляет собой синтетическую бинарную задачу классификации из $n = 3000$ объектов, описанных $d = 8$ признаками. При генерации данных задаётся, что 3 признака являются информативными, ещё 2 - избыточными, то есть линейно связаны с информативными и не добавляют существенно новой информации. Классы разделены с параметром `class_sep` $= 1.2$, а в разметку внесён небольшой уровень шума: доля случайных инверсий меток составляет `flip_y` $= 0.02$. Чтобы сделать зависимость целевого признака от части координат нелинейной, дополнительно выполняются преобразования признаков: первая координата заменяется на $x_0 \leftarrow \sin(x_0) + 0.1x_1$, а третья - на $x_2 \leftarrow x_2^2$. В результате DS-A сочетает линейно-информативную структуру, избыточность и локальные нелинейности, что позволяет оценить устойчивость методов редукции и отбора признаков при уменьшении размерности. В экспериментах для DS-A рассматривались размеры редукции $q \in \{2, 3, 5\}$.

Таблица 1. Результаты на DS-A (HGBT, $q = 2$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.970565	0.974506	-0.017788	-0.014287	0.285843	0.200469
MLP_unar	0.986348	0.986826	-0.002006	-0.001966	117.725034	0.204162
PCA	0.978915	0.975961	-0.009438	-0.012831	0.001453	0.199110
PLS	0.980662	0.979784	-0.007691	-0.009009	0.001888	0.199345
UMAP_sup	0.980732	0.980909	-0.007621	-0.007884	11.710310	0.199346

Таблица 2. Результаты на DS-A (HGBT, $q = 3$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.987754	0.988550	-0.000599	-0.000243	0.300731	0.211273
MLP_unar	0.985921	0.987263	-0.002432	-0.001529	106.325773	0.323044
PCA	0.982696	0.981152	-0.005657	-0.007641	0.001158	0.210256
PLS	0.981381	0.981300	-0.006972	-0.007492	0.002058	0.206862
UMAP_sup	0.981740	0.978744	-0.006613	-0.010049	12.419136	0.215452

Таблица 3. Результаты на DS-A (HGBT, $q = 5$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.986532	0.987296	-0.001821	-0.001497	0.280258	0.223886
MLP_unar	0.988919	0.990394	0.000566	0.001601	75.101636	0.231676
PCA	0.986327	0.983510	-0.002027	-0.005282	0.001225	0.236653
PLS	0.983284	0.983515	-0.005069	-0.005277	0.002653	0.226112
UMAP_sup	0.981842	0.975839	-0.006511	-0.012954	12.897769	0.223852

Таблица 4. Результаты на DS-A (MLP, $q = 2$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.971751	0.975836	-0.014325	-0.010527	0.285843	0.137024
MLP_unar	0.984589	0.985793	-0.001487	-0.000570	117.725034	0.205935
PCA	0.984104	0.984548	-0.001971	-0.001815	0.001453	0.261257
PLS	0.984433	0.984184	-0.001642	-0.002179	0.001888	0.156200
UMAP_sup	0.977611	0.975042	-0.008465	-0.011321	11.710310	0.147092

Таблица 5. Результаты на DS-A (MLP, $q = 3$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.983300	0.984784	-0.002776	-0.001579	0.300731	0.273864
MLP_unar	0.984815	0.986107	-0.001260	-0.000256	106.325773	0.827545
PCA	0.984529	0.984277	-0.001547	-0.002086	0.001158	0.191964
PLS	0.984493	0.984251	-0.001582	-0.002112	0.002058	0.157493
UMAP_sup	0.979955	0.979153	-0.006120	-0.007210	12.419136	0.152262

Таблица 6. Результаты на DS-A (MLP, $q = 5$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.986713	0.988009	0.000638	0.001646	0.280258	0.173197
MLP_unar	0.981584	0.982671	-0.004491	-0.003692	75.101636	0.188049
PCA	0.984573	0.983518	-0.001502	-0.002845	0.001225	0.255150
PLS	0.982666	0.983012	-0.003409	-0.003351	0.002653	0.166903
UMAP_sup	0.980662	0.979699	-0.005413	-0.006664	12.897769	0.209122

На DS-A метод MLP_unar показывает наилучшее качество для HGBT при $q = 5$ и для MLP при $q \in \{2, 3\}$, однако различия с ближайшими конкурентами в этих режимах находятся на уровне тысячных. При $q = 5$ и базовой модели MLP метод уступает

HSIC_Lasso и линейным редукторам, однако разница по ROC AUC опять же порядка 0.005 и остаётся в пределах сопоставимого качества; такое поведение может зависеть от статистической корректности оценки, а также от настройки гиперпараметров унарной части.

6.2 Синтетический набор данных DS-B

Набор данных DS-B представляет собой синтетическую бинарную задачу классификации повышенной размерности, содержащую $n = 6000$ объектов, описанных $d = 30$ признаками. При генерации данных задаётся, что 5 признаков являются информативными, ещё 5 - избыточными, а распределение классов существенно несбалансировано: доля объектов положительного класса составляет около 25%. Разделимость классов умеренная (`class_sep` $= 0.8$), при этом в разметку добавлен небольшой уровень шума с долей инверсий меток `flip_y` $= 0.01$.

Для усложнения структуры зависимости целевого признака от признакового пространства в набор данных включены как нелинейные преобразования, так и взаимодействия между координатами. В частности, вводятся признаки вида $x_{10} \leftarrow \sin(x_0)\, x_1$, $x_{11} \leftarrow x_2^2$ и $x_{12} \leftarrow \exp(-x_3^2)$, а также набор слабо зашумлённых копий первых информативных признаков, размещённых в координатах 20:25. Такая конструкция моделирует типичную для прикладных табличных данных ситуацию, в которой одновременно присутствуют информативные и избыточные признаки, коррелированные группы переменных и смешанная линейно-нелинейная структура зависимостей.

В экспериментах для DS-B рассматривались размеры редукции $q \in \{5, 10, 15\}$.

Таблица 7. Результаты на DS-B (HGBT, $q = 5$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
Ensembleunar	0.943656	0.872110	-0.014764	-0.033804	1462.708774	0.338188
HSIC_Lasso	0.954001	0.896141	-0.004419	-0.009773	0.605554	0.337242
PLS	0.946973	0.885809	-0.011447	-0.020106	0.007177	0.297453
PCA	0.677136	0.429959	-0.281284	-0.475956	0.002488	0.285192
UMAP_sup	0.745107	0.527121	-0.213313	-0.378794	16.514097	0.295961

Таблица 8. Результаты на DS-B (HGBT, $q = 10$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
Ensembleunar	0.959129	0.906669	0.000708	0.000754	1466.362532	0.340065
PLS	0.954915	0.902382	-0.003506	-0.003532	0.012408	0.319568
HSIC_Lasso	0.944929	0.878895	-0.013491	-0.027020	0.699972	0.317398
PCA	0.876723	0.688229	-0.081698	-0.217685	0.002740	0.337652
UMAP_sup	0.775408	0.567177	-0.183012	-0.338737	20.492583	0.341360

Таблица 9. Результаты на DS-B (HGBT, $q = 15$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
Ensembleunar	0.955862	0.901995	-0.002558	-0.003919	1469.874476	0.344023
PLS	0.952853	0.899741	-0.005567	-0.006173	0.016798	0.335751
HSIC_Lasso	0.942160	0.876210	-0.016260	-0.029704	0.936248	0.365223
PCA	0.889813	0.716948	-0.068608	-0.188967	0.003392	0.331456
UMAP_sup	0.782580	0.573703	-0.175840	-0.332212	23.408102	0.360922

Таблица 10. Результаты на DS-B (MLP, $q = 5$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
Ensembleunar	0.960336	0.914073	0.028133	0.049714	1462.708774	1.023030
HSIC_Lasso	0.953842	0.906015	0.021639	0.041656	0.605554	0.630133
PLS	0.939943	0.872374	0.007740	0.008015	0.007177	0.712632
PCA	0.687974	0.448327	-0.244230	-0.416032	0.002488	0.397214
UMAP_sup	0.549548	0.289236	-0.382655	-0.575123	16.514097	0.179547

Таблица 11. Результаты на DS-B (MLP, $q = 10$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
Ensembleunar	0.964891	0.921808	0.032688	0.057449	1466.362532	1.074756
PLS	0.953977	0.900823	0.021773	0.036464	0.012408	0.645237
HSIC_Lasso	0.939133	0.876846	0.006929	0.012487	0.699972	0.762155
PCA	0.868926	0.681057	-0.063277	-0.183303	0.002740	0.695374
UMAP_sup	0.484301	0.249009	-0.447902	-0.615350	20.492583	0.178839

Таблица 12. Результаты на DS-B (MLP, $q = 15$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
Ensembleunar	0.944871	0.888826	0.012668	0.024467	1469.874476	0.752957
PLS	0.944392	0.884113	0.012189	0.019754	0.016798	0.592478
HSIC_Lasso	0.935508	0.870047	0.003305	0.005688	0.936248	1.190793
PCA	0.893645	0.735038	-0.038558	-0.129321	0.003392	0.747590
UMAP_sup	0.548465	0.290475	-0.383738	-0.573884	23.408102	0.178606

На DS-B метод Ensembleunar демонстрирует наилучшее качество среди сравниваемых подходов для обеих последующих моделей (HGBT и MLP) при $q \in \{10, 15\}$, а для MLP также при $q = 5$. При $q = 5$ и HGBT метод уступает HSIC_Lasso, однако

разница по ROC AUC порядка 0.01 остаётся в пределах сопоставимого качества. Существенным практическим ограничением является высокая вычислительная стоимость редуктора по времени.

6.3 Реальный набор данных DS-A-real

В качестве первого реального набора данных используется набор данных Rice (Cammeo and Osmancik), представляющий собой задачу бинарной классификации двух сортов риса на основе морфологических характеристик отдельных зёрен. Каждый объект описывается $d = 7$ компактными геометрическими признаками, полученными в результате обработки изображений зёрен, включая характеристики формы, размеров и контура. Эти признаки хорошо интерпретируются с точки зрения предметной области и при этом содержат достаточную различительную информацию между сортами, что делает данный набор данных удобным тестовым примером для методов отбора признаков и понижения размерности в условиях относительно малой размерности и умеренной коррелированности координат. Описание набора данных и исходная постановка задачи приведены в работе [13]. В экспериментах для набора данных DS-A-real рассматривались размеры редукции $q \in \{2, 3, 5\}$.

Таблица 13. Результаты на DS-A-real (HGBT, $q = 2$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.973615	0.979031	0.000137	-0.000615	0.323406	0.200964
MLP_unar	0.968427	0.974582	-0.005051	-0.005064	312.459187	0.233460
PCA	0.969744	0.976321	-0.003733	-0.003325	0.001330	0.202325
PLS	0.973056	0.979334	-0.000421	-0.000312	0.002072	0.204862
UMAP_sup	0.969421	0.975281	-0.004057	-0.004364	17.879416	0.203783

Таблица 14. Результаты на DS-A-real (HGBT, $q = 3$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.973096	0.979010	-0.000381	-0.000636	0.326939	0.211409
MLP_unar	0.972266	0.979232	-0.001212	-0.000413	342.233841	0.296109
PCA	0.971694	0.977297	-0.001784	-0.002349	0.001181	0.216057
PLS	0.973231	0.979678	-0.000246	0.000032	0.002241	0.211898
UMAP_sup	0.970269	0.977100	-0.003208	-0.002546	17.608202	0.213535

Таблица 15. Результаты на DS-A-real (HGBT, $q = 5$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.973252	0.979620	-0.000226	-0.000026	0.332584	0.223249
MLP_unar	0.973243	0.979753	-0.000235	0.000107	159.219546	0.223269
PCA	0.972277	0.977955	-0.001200	-0.001691	0.001325	0.239144
PLS	0.974208	0.980271	0.000730	0.000625	0.002769	0.228442
UMAP_sup	0.970365	0.976384	-0.003113	-0.003261	18.400578	0.237983

Таблица 16. Результаты на DS-A-real (MLP, $q = 2$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.977700	0.982612	0.000367	0.000213	0.323406	0.331590
MLP_unar	0.973095	0.977879	-0.004238	-0.004520	312.459187	0.301596
PCA	0.976083	0.981470	-0.001250	-0.000929	0.001330	0.241261
PLS	0.977713	0.982632	0.000380	0.000233	0.002072	0.224398
UMAP_sup	0.965429	0.969389	-0.011904	-0.013010	17.879416	0.459461

Таблица 17. Результаты на DS-A-real (MLP, $q = 3$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.977711	0.983051	0.000379	0.000652	0.326939	0.280160
MLP_unar	0.977507	0.983069	0.000174	0.000670	342.233841	0.347544
PCA	0.977507	0.982776	0.000174	0.000378	0.001181	0.186016
PLS	0.977316	0.982379	-0.000017	-0.000020	0.002241	0.200615
UMAP_sup	0.968809	0.973760	-0.008524	-0.008639	17.608202	0.322675

Таблица 18. Результаты на DS-A-real (MLP, $q = 5$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.976299	0.981623	-0.001034	-0.000776	0.332584	0.205971
MLP_unar	0.976279	0.981302	-0.001054	-0.001097	159.219546	0.247190
PCA	0.975135	0.980608	-0.002198	-0.001791	0.001325	0.277232
PLS	0.978093	0.983197	0.000760	0.000798	0.002769	0.298271
UMAP_sup	0.971236	0.976596	-0.006097	-0.005803	18.400578	0.239608

На DS-A-real метод MLP_unar находится на уровне HSIC_Lasso и близок к линейным редукторам (PCA/PLS), а отставание от лучшего результата, как правило,

составляет тысячные. При $q = 2$ наблюдается более заметный разрыв по сравнению с лидерами, однако он остаётся в пределах ≈ 0.005 по ROC AUC, то есть качество остаётся сопоставимым; различия могут зависеть от режима валидации и настройки гиперпараметров.

6.4 Реальный набор данных DS-B-real

В качестве второго реального набора данных используется набор данных Phishing Websites, который включает $d = 30$ табличных признаков, характеризующих URL и свойства веб-страниц, и предназначен для задачи бинарной классификации фишинговых сайтов. Признаки в наборе данных дискретны и отражают различные аспекты, такие как длина URL, использование поддоменов, наличие специальных символов и другие эвристики, позволяющие отличить фишинговые сайты от легитимных. Этот набор данных моделирует практическую задачу в области кибербезопасности, где разделимость классов достигается за счет комбинации различных признаков. Описание набора данных приведено в работе [14]. В экспериментах для набора данных DS-B-real использовались размеры редукции $q \in \{5, 10, 15\}$.

Таблица 19. Результаты на DS-B-real (HGBT, $q = 5$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
PLS	0.989753	0.991517	-0.006066	-0.004977	0.012622	0.319731
PCA	0.986859	0.988822	-0.008960	-0.007672	0.003560	0.316847
HSIC_Lasso	0.981596	0.982182	-0.014223	-0.014313	1.201798	0.320936
Ensembleunar	0.970599	0.968780	-0.025220	-0.027714	434.225702	0.294692
UMAP_sup	0.967280	0.972473	-0.028539	-0.024021	21.764809	0.330768

Таблица 20. Результаты на DS-B-real (HGBT, $q = 10$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
PCA	0.991934	0.993346	-0.003885	-0.003148	0.012073	0.345465
PLS	0.991421	0.992924	-0.004398	-0.003570	0.019920	0.336661
HSIC_Lasso	0.990009	0.991589	-0.005810	-0.004906	1.393820	0.335443
Ensembleunar	0.981221	0.983066	-0.014597	-0.013429	434.457498	0.319707
UMAP_sup	0.971977	0.976625	-0.023842	-0.019869	23.361637	0.351070

Таблица 21. Результаты на DS-B-real (HGBT, $q = 15$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
PLS	0.993227	0.994478	-0.002592	-0.002016	0.026245	0.369544
PCA	0.993068	0.994353	-0.002751	-0.002141	0.012378	0.378682
HSIC_Lasso	0.990512	0.992107	-0.005306	-0.004388	1.392486	0.334822
Ensembleunar	0.987567	0.989203	-0.008252	-0.007291	438.116849	0.336248
UMAP_sup	0.972000	0.976483	-0.023818	-0.020011	31.216013	0.359395

Таблица 22. Результаты на DS-B-real (MLP, $q = 5$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
PLS	0.978977	0.982998	-0.015351	-0.012374	0.012622	0.925834
HSIC_Lasso	0.977065	0.978955	-0.017262	-0.016418	1.201798	1.208000
PCA	0.970663	0.975464	-0.023664	-0.019909	0.003560	0.612417
Ensembleunar	0.964499	0.963916	-0.029828	-0.031457	434.225702	0.618334
UMAP_sup	0.913640	0.930414	-0.080688	-0.064959	21.764809	2.222381

Таблица 23. Результаты на DS-B-real (MLP, $q = 10$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
HSIC_Lasso	0.984309	0.987019	-0.010019	-0.008353	1.393820	1.063420
PCA	0.984345	0.987509	-0.009982	-0.007864	0.012073	1.321070
PLS	0.983363	0.986768	-0.010964	-0.008605	0.019920	1.041679
Ensembleunar	0.975694	0.978179	-0.018633	-0.017194	434.457498	1.045043
UMAP_sup	0.917282	0.933396	-0.077045	-0.061977	23.361637	1.782164

Таблица 24. Результаты на DS-B-real (MLP, $q = 15$)

Метод	ROC AUC	PR AUC	ΔROC AUC	ΔPR AUC	$t_{\text{ред}}$, с	$t_{\text{мод}}$, с
PCA	0.988880	0.991176	-0.005447	-0.004197	0.012378	1.876479
PLS	0.988173	0.990636	-0.006155	-0.004736	0.026245	1.738431
HSIC_Lasso	0.986386	0.988938	-0.007942	-0.006435	1.392486	1.553514
Ensembleunar	0.982562	0.984832	-0.011765	-0.010540	438.116849	0.996738
UMAP_sup	0.900993	0.920600	-0.093335	-0.074773	31.216013	1.602632

На DS-B-real метод Ensembleunar уступает HSIC_Lasso, однако разница по ROC AUC во всех рассмотренных режимах не превышает ≈ 0.02, то есть результаты остаются сопоставимыми по качеству и могут зависеть от режима настройки гиперпараметров

и статистических свойств процедуры оценки информативности. Однако при этом вычислительная стоимость редуктора существенно выше.

Заключение

В работе предложены и исследованы два новых метода оценки информативности признаков табличных данных в задаче бинарной классификации, основанные на идее унарной классификации с добавлением фонового класса, равномерно распределённого на компакте. Такая постановка фиксирует корректное поведение модели вне носителя данных и формирует естественную зону отказа, что делает последующие оценки информативности более устойчивыми.

Первый метод представляет собой адаптацию схемы последовательного исключения признаков с переобучением моделей к унарной постановке и использует унарные метрики разделимости, учитывающие области однозначного решения, наложения и отказа. Второй метод сочетает ансамбли случайных подпространств с унарными классификаторами и обновлением распределения важностей координат на основе аддитивных объяснений Шепли, что позволяет частично преодолевать проклятие размерности за счёт обучения на подпространствах малой размерности.

Экспериментальное исследование проведено на четырёх наборах данных: двух синтетических и двух реальных. Предложенные методы сравнивались с базовыми подходами понижения размерности и отбора признаков, включая метод главных компонент, метод частичных наименьших квадратов, проекцию многообразия в постановке с учителем, а также отбор признаков по критерию независимости Гильберта-Шмидта с разреживающей регуляризацией. Качество оценивалось по влиянию редукции на последующие модели классификации в терминах ROC AUC и PR AUC.

Полученные результаты показывают конкурентоспособность предложенных подходов. В дальнейших работах планируется улучшение результатов за счёт систематического подбора гиперпараметров унарной части, а также за счёт проведения более широких экспериментов на дополнительных реальных наборах данных и в различных режимах дисбаланса классов. Отдельным направлением является усиление теоретического обоснования предложенных методов. Кроме того, предполагается модифицировать критерии выбора моделей и порогов в унарной постановке так, чтобы перенести предложенные идеи на многоклассовый случай.

Таким образом, уже на текущем этапе предложенные методы формируют работоспособный и интерпретируемый инструментарий оценки информативности признаков в табличной классификации на базе унарного подхода с зоной отказа, демонстрируя качество на уровне сильных базовых методов и задавая направления для дальнейшего повышения точности, устойчивости и практической эффективности.

Список использованных источников

[1] Lukyanov K. S. et al. Extrapolation of the Bayesian classifier with an unknown support of the two-class mixture distribution //Uspekhi Matematicheskikh Nauk. – 2024. – T. 79. – №. 6. – C. 57-82.

[2] Pearson, K. On lines and planes of closest fit to systems of points in space. Philosophical Magazine, Series 6, 2(11), 559–572 (1901).

[3] Hotelling, H. Analysis of a complex of statistical variables into principal components. Journal of Educational Psychology, 24, 417–441, 498–520 (1933).

[4] Wold, S., Ruhe, A., Wold, H., Dunn, W. J. The collinearity problem in linear regression. The partial least squares approach to generalized inverses. SIAM Journal on Scientific and Statistical Computing, 5(3), 735–743 (1984).

[5] McInnes, L., Healy, J., Melville, J. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. arXiv:1802.03426 (2018).

[6] McInnes, L., Healy, J., Saul, N., Grossberger, L. UMAP: Uniform Manifold Approximation and Projection. Journal of Open Source Software, 3(29), 861 (2018).

[7] Gretton, A., Bousquet, O., Smola, A., Schölkopf, B. Measuring Statistical Dependence with Hilbert-Schmidt Norms. In: Algorithmic Learning Theory (ALT), 63–77 (2005).

[8] Yamada, M., Jitkrittum, W., Sigal, L., Xing, E. P., Sugiyama, M. High-Dimensional Feature Selection by Feature-Wise Kernelized Lasso. Neural Computation, 26(1), 185–207 (2014).

[9] Н. А. Елисеев, А. И. Перминов, Д. Ю. Турдаков, Сходимость многослойного персептрона к гистограммной байесовской регрессии, УМН, 80:6(486) (2025), 45–72.

[10] Liu J., Danait N., Hu S., Sengupta S. A leave-one-feature-out wrapper method for feature selection in data classification // Proceedings of the 2013 6th International Conference on Biomedical Engineering and Informatics (BMEI). 2013. DOI: 10.1109/BMEI.2013.6747021.

[11] Tian Y., Feng Y. RaSE: Random subspace ensemble classification // Journal of Machine Learning Research. — 2021. — Т. 22. — № 45. — С. 1–93.

[12] Lundberg S. M., Lee S.-I. A Unified Approach to Interpreting Model Predictions // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 4765–4774.

[13] Rice (Cammeo and Osmancik) [Dataset]. UCI Machine Learning Repository, 2019. DOI: 10.24432/C5MW4Z.

[14] Mohammad R., McCluskey L. Phishing Websites [Dataset]. UCI Machine Learning Repository, 2012. DOI: 10.24432/C51W2X.

ПРИЛОЖЕНИЯ

Ниже приведен код всех экспериментов и методов, описанных в работе.

ПРИЛОЖЕНИЕ А

Ниже приведен код, относящийся к LOFO методу и частично используемый для RaSE метода.

```
import numpy as np
import pandas as pd
from time import perf_counter
from typing import Any, Dict, List

from sklearn.datasets import make_classification
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score, average_precision_score,
    precision_recall_fscore_support
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import HistGradientBoostingClassifier

from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression
from umap import UMAP

from pyHSICLasso import HSICLasso

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset, random_split,
     Subset, Dataset, ConcatDataset
from sklearn.model_selection import train_test_split
from typing import Tuple, Optional, Iterable
import random
import shap
import copy
import pickle
from sklearn.model_selection import StratifiedShuffleSplit
from ucimlrepo import fetch_ucirepo
```

```python

def compute_feature_bounds(dataset: Dataset, pad: float = 0.25):
    if isinstance(dataset, torch.utils.data.TensorDataset):
        X = dataset.tensors[0].to(dtype=torch.float32)
        fmin = X.amin(dim=0)
        fmax = X.amax(dim=0)
        span = fmax - fmin
        return fmin - pad * span, fmax + pad * span

    if isinstance(dataset, torch.utils.data.Subset) and isinstance(dataset.dataset, torch.utils.data.TensorDataset):
        Xfull = dataset.dataset.tensors[0]
        idx = torch.as_tensor(dataset.indices, device=Xfull.device, dtype=torch.long)
        X = Xfull.index_select(0, idx).to(dtype=torch.float32)
        fmin = X.amin(dim=0)
        fmax = X.amax(dim=0)
        span = fmax - fmin
        return fmin - pad * span, fmax + pad * span

    xs = [dataset[i][0] for i in range(len(dataset))]
    if len(xs) == 0:
        raise ValueError("compute_feature_bounds: empty dataset")
    device = xs[0].device if torch.is_tensor(xs[0]) else torch.device("cpu")
    xs = [x.to(device) if torch.is_tensor(x) else torch.as_tensor(x, device=device) for x in xs]
    X = torch.stack(xs, dim=0).to(dtype=torch.float32)
    fmin = X.min(dim=0).values
    fmax = X.max(dim=0).values
    span = fmax - fmin
    return fmin - pad * span, fmax + pad * span

class UniformNoiseDataset(Dataset):
    def __init__(
        self,
        feature_min: torch.Tensor,
        feature_max: torch.Tensor,
        n: int,
        label: int = 0,
        dtype: torch.dtype | None = None
    ):
        assert feature_min.shape == feature_max.shape
```

```python

        self.dtype = dtype if dtype is not None else torch.float32

        feature_min = feature_min.to(dtype=self.dtype)
        feature_max = feature_max.to(device=feature_min.device, dtype=self.dtype)

        self.feature_min = feature_min
        self.feature_max = feature_max
        self.n = int(n)
        self.label = int(label)

    def __len__(self) -> int:
        return self.n

    def __getitem__(self, idx: int):
        u = torch.rand_like(self.feature_min, dtype=self.dtype)
        x = self.feature_min + u * (self.feature_max - self.feature_min)
        y = torch.tensor(self.label, dtype=torch.long, device=x.device)
        return x, y

class LabelMappedSubset(Dataset):
    def __init__(self, subset: Subset, label_map: dict[int, int]):
        self.subset = subset
        self.label_map = label_map

    def __getitem__(self, idx):
        x, y = self.subset[idx]

        if torch.is_tensor(y):
            y_int = int(y.detach().cpu().item())
        else:
            y_int = int(y)

        y_mapped = self.label_map.get(y_int, y_int)
        return x, torch.tensor(y_mapped, dtype=y.dtype, device=y.device)

    def __len__(self):
        return len(self.subset)

```

```python

class Abs(nn.Module):
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.abs(x)

class MLP(nn.Module):
    def __init__(self, d_in: int, d_hidden: int = 10, n_hidden_layers
       : int = 2):

        super().__init__()

        layers = []

        layers.append(nn.Linear(d_in, d_hidden))
        layers.append(Abs())


        for _ in range(n_hidden_layers - 1):
            layers.append(nn.Linear(d_hidden, d_hidden))
            layers.append(Abs())

        layers.append(nn.Linear(d_hidden, 1))

        self.net = nn.Sequential(*layers)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)

def train_one_model(
    model: nn.Module,
    dataset_pos_as_1: Dataset,
    feature_min: torch.Tensor,
    feature_max: torch.Tensor,
    num_epochs: int = 20,
    batch_size: int = 128,
    lr: float = 3e-3,
    weight_decay: float = 1e-2,
    seed: int = 0,
):
    """
    Обучает модель различать:
      - реальные (label=1)
      - шумовые (label=0)
```

```
    c MSELoss.
    """
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

    device = next(model.parameters()).device
    criterion = nn.MSELoss()
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=
       weight_decay)

    if isinstance(dataset_pos_as_1, TensorDataset):
        X_real = dataset_pos_as_1.tensors[0].to(device=device, dtype=
           torch.float32)
        n = int(X_real.shape[0])
        if n == 0:
            raise ValueError("train_one_model: empty TensorDataset")

        fmin = feature_min.to(device=device, dtype=torch.float32)
        fmax = feature_max.to(device=device, dtype=torch.float32)
        span = (fmax - fmin)
        d = int(X_real.shape[1])

        for _ in range(num_epochs):
            model.train()
            perm = torch.randperm(n, device=device)

            for start in range(0, n, batch_size):
                idx = perm[start:start + batch_size]
                xb_real = X_real.index_select(0, idx)


                u = torch.rand(xb_real.shape[0], d, device=device,
                   dtype=torch.float32)
                xb_noise = fmin.unsqueeze(0) + u * span.unsqueeze(0)

                xb = torch.cat([xb_real, xb_noise], dim=0)
                yb = torch.cat(
                    [
                        torch.ones(xb_real.shape[0], device=device,
                           dtype=torch.float32),
                        torch.zeros(xb_noise.shape[0], device=device,
                            dtype=torch.float32),
```

```
                    ],
                    dim=0,
                ).unsqueeze(1)

                pred = model(xb)
                loss = criterion(pred, yb)

                opt.zero_grad(set_to_none=True)
                loss.backward()
                opt.step()

        return model


    n_real = len(dataset_pos_as_1)
    if n_real <= 0:
        raise ValueError("train_one_model: empty dataset_pos_as_1")

    n_noise = max(1, n_real)

    x0, _ = dataset_pos_as_1[0]
    data_device = x0.device if torch.is_tensor(x0) else device

    fmin = feature_min.detach().to(device=data_device, dtype=torch.
       float32)
    fmax = feature_max.detach().to(device=data_device, dtype=torch.
       float32)

    noise_ds = UniformNoiseDataset(fmin, fmax, n_noise, label=0,
       dtype=torch.float32)
    train_ds = ConcatDataset([dataset_pos_as_1, noise_ds])
    loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True
       , drop_last=False)

    for _ in range(num_epochs):
        model.train()
        for xb, yb in loader:
            xb = xb.to(device=device, dtype=torch.float32)
            yb = yb.to(device=device).unsqueeze(-1).to(dtype=torch.
               float32)

            pred = model(xb)
            loss = criterion(pred, yb)
```

```python

            opt.zero_grad(set_to_none=True)
            loss.backward()
            opt.step()

    return model

def model_train(
    d_hidden: int,
    n_hidden_layers: int,
    train_data: Dataset,
    num_epochs: int,
    batch_size: int = 128,
    seed: int = 0,
    output: bool = False,
    device: str | torch.device | None = None,
    feature_min: torch.Tensor | None = None,
    feature_max: torch.Tensor | None = None,
):
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

    if isinstance(train_data, TensorDataset):
        X_all, y_all = train_data.tensors


        data_device = X_all.device
        dev = data_device if device is None else torch.device(device)

        X_all = X_all.to(device=dev, dtype=torch.float32)
        y_all = y_all.to(device=dev, dtype=torch.long)

        d_in = int(X_all.shape[1])

        if feature_min is None or feature_max is None:
            fmin, fmax = compute_feature_bounds(TensorDataset(X_all,
                y_all))
        else:
            fmin, fmax = feature_min, feature_max
        fmin = fmin.to(device=dev, dtype=torch.float32)
        fmax = fmax.to(device=dev, dtype=torch.float32)

```

```
        pos_mask = (y_all == 1)
        neg_mask = (y_all == -1)
        X_pos = X_all[pos_mask]
        X_neg = X_all[neg_mask]

        if X_pos.numel() == 0 or X_neg.numel() == 0:
            raise ValueError("model_train: empty class after split (pos or neg)")

        model_pos = MLP(d_in=d_in, d_hidden=d_hidden, n_hidden_layers=n_hidden_layers).to(dev)
        model_neg = MLP(d_in=d_in, d_hidden=d_hidden, n_hidden_layers=n_hidden_layers).to(dev)

        ds_pos = TensorDataset(X_pos, torch.ones(X_pos.size(0), device=dev, dtype=torch.long))
        ds_neg = TensorDataset(X_neg, torch.ones(X_neg.size(0), device=dev, dtype=torch.long))

        model_pos = train_one_model(
            model_pos, ds_pos, fmin, fmax,
            num_epochs=num_epochs, batch_size=batch_size, seed=seed
        )
        model_neg = train_one_model(
            model_neg, ds_neg, fmin, fmax,
            num_epochs=num_epochs, batch_size=batch_size, seed=seed + 1
        )
        return model_pos, model_neg

    ys: list[int] = []
    for i in range(len(train_data)):
        y = train_data[i][1]
        if torch.is_tensor(y):
            ys.append(int(y.detach().cpu().item()))
        else:
            ys.append(int(y))
    labels = torch.tensor(ys, dtype=torch.long)

    pos_idx = (labels == 1).nonzero(as_tuple=True)[0].tolist()
    neg_idx = (labels == -1).nonzero(as_tuple=True)[0].tolist()

    train_pos = Subset(train_data, pos_idx)
```

```
    train_neg = Subset(train_data, neg_idx)
    train_neg_as_pos = LabelMappedSubset(train_neg, {-1: 1})

    x0, _ = train_data[0]
    d_in = x0.shape[-1]
    data_device = x0.device if torch.is_tensor(x0) else torch.device("cpu")

    dev = data_device if device is None else torch.device(device)

    fmin, fmax = compute_feature_bounds(train_data)
    fmin = fmin.to(dev)
    fmax = fmax.to(dev)

    model_pos = MLP(d_in=d_in, d_hidden=d_hidden, n_hidden_layers=n_hidden_layers).to(dev)
    model_neg = MLP(d_in=d_in, d_hidden=d_hidden, n_hidden_layers=n_hidden_layers).to(dev)

    model_pos = train_one_model(
        model_pos, train_pos, fmin, fmax,
        num_epochs=num_epochs, batch_size=batch_size, seed=seed
    )
    model_neg = train_one_model(
        model_neg, train_neg_as_pos, fmin, fmax,
        num_epochs=num_epochs, batch_size=batch_size, seed=seed + 1
    )
    return model_pos, model_neg

def metrics_from_sheet(X, y, model_pos, model_neg, beta_pos=0.1, beta_neg=0.1):
    device = next(model_pos.parameters()).device
    X = X.to(device=device, dtype=torch.float32)
    y = y.to(device=device)

    model_pos.eval()
    model_neg.eval()
    with torch.no_grad():
        s1 = model_pos(X).flatten()
        s2 = model_neg(X).flatten()

    pos = (y == 1)
    neg = (y == -1)
```

```
    n1 = int(pos.sum().item())
    n2 = int(neg.sum().item())

    n02_1 = int((((s1 >= beta_pos) & (s2 <  beta_neg)) & pos).sum().item())
    n12_1 = int((((s1 >= beta_pos) & (s2 >= beta_neg)) & pos).sum().item())

    n02_2 = int((((s2 >= beta_neg) & (s1 <  beta_pos)) & neg).sum().item())
    n12_2 = int((((s2 >= beta_neg) & (s1 >= beta_pos)) & neg).sum().item())

    a1 = n02_1 / n1 if n1 else 0.0
    a2 = n02_2 / n2 if n2 else 0.0
    F12 = 2 * a1 * a2 / (a1 + a2 + 1e-12)

    b1 = n12_1 / n1 if n1 else 0.0
    b2 = n12_2 / n2 if n2 else 0.0
    G12 = 2 * b1 * b2 / (b1 + b2 + 1e-12)

    return dict(
        n1=n1, n2=n2,
        n02_1=n02_1, n12_1=n12_1,
        n02_2=n02_2, n12_2=n12_2,
        a1=a1, a2=a2, F12=F12,
        b1=b1, b2=b2, G12=G12
    )


def pick_beta(scores_pos: torch.Tensor,
              scores_noise: torch.Tensor,
              scores_neg: Optional[torch.Tensor] = None,
              n_quantiles: int = 19) -> Tuple[float, float]:

    s_pos = scores_pos.detach().flatten()
    s_no  = scores_noise.detach().flatten()
    s_neg = scores_neg.detach().flatten() if (scores_neg is not None) else None

    if s_pos.numel() == 0:
        return 0.0, -1.0

```

```
    device = s_pos.device

    qs = torch.linspace(0.05, 0.95, steps=n_quantiles, device=device)

    if s_neg is None:
        merged = torch.cat([s_pos, s_no])
    else:
        merged = torch.cat([s_pos, s_no, s_neg])

    pool = torch.quantile(merged, qs)

    best_J, best_b = -1.0, float("nan")

    for b_tensor in pool:
        b = float(b_tensor.item())

        tpr = (s_pos >= b).float().mean().item() if s_pos.numel()
            else 0.0

        if s_neg is None:
            tnr = (s_no < b).float().mean().item() if s_no.numel()
                else 0.0
        else:
            tnr_no  = (s_no  < b).float().mean().item() if s_no.numel
                ()  else 0.0
            tnr_neg = (s_neg < b).float().mean().item() if s_neg.
                numel() else 0.0
            tnr = 0.5 * (tnr_no + tnr_neg)

        J = tpr + tnr - 1.0
        if J > best_J:
            best_J, best_b = J, b

    return best_b, best_J


def choose_coord(
    X: torch.Tensor,
    y: torch.Tensor,
    d_hidden: int,
    n_hidden_layers: int,
    num_epochs: int,
    batch_size: int = 128,
```

```
    seed: int = 0,
    output: bool = False,
):

    if torch.is_tensor(y) and y.device != X.device:
        y = y.to(X.device)

    n_features = X.shape[1]


    idx_np = torch.arange(len(y)).detach().cpu().numpy()
    y_np = y.detach().cpu().numpy()

    train_idx_np, val_idx_np = train_test_split(
        idx_np, test_size=0.2, random_state=seed, stratify=y_np
    )


    train_idx = torch.as_tensor(train_idx_np, dtype=torch.long,
       device=X.device)
    val_idx   = torch.as_tensor(val_idx_np,   dtype=torch.long,
       device=X.device)


    X_tr, y_tr   = X[train_idx], y[train_idx]
    X_val, y_val = X[val_idx],   y[val_idx]

    fmin_all, fmax_all = compute_feature_bounds(TensorDataset(X_tr,
       y_tr))
    fmin_all = fmin_all.to(X_tr.device)
    fmax_all = fmax_all.to(X_tr.device)

    rows = []
    best_idx, best_key = None, -float("inf")

    for i in range(n_features):

        X_tr_upd  = torch.cat((X_tr[:, :i],  X_tr[:, i+1:]),  dim=1)
        X_val_upd = torch.cat((X_val[:, :i], X_val[:, i+1:]), dim=1)

        fmin_upd = torch.cat((fmin_all[:i], fmin_all[i+1:]))
        fmax_upd = torch.cat((fmax_all[:i], fmax_all[i+1:]))

```

```
        n_pos_val = int((y_val == 1).sum().item())
        n_neg_val = int((y_val == -1).sum().item())
        n_noise_for_pos = max(1, n_pos_val)
        n_noise_for_neg = max(1, n_neg_val)

        noise_pos_ds = UniformNoiseDataset(fmin_upd, fmax_upd,
           n_noise_for_pos, label=0, dtype=torch.float32)
        noise_neg_ds = UniformNoiseDataset(fmin_upd, fmax_upd,
           n_noise_for_neg, label=0, dtype=torch.float32)

        X_noise_pos = torch.stack([noise_pos_ds[j][0] for j in range(
           len(noise_pos_ds))], dim=0)
        X_noise_neg = torch.stack([noise_neg_ds[j][0] for j in range(
           len(noise_neg_ds))], dim=0)

        train_data_upd = TensorDataset(X_tr_upd, y_tr)

        model_pos, model_neg = model_train(
            train_data=train_data_upd,
            d_hidden=d_hidden,
            n_hidden_layers=n_hidden_layers,
            num_epochs=num_epochs,
            batch_size=batch_size,
            seed=seed,
            output=output,
        )

        model_pos.eval()
        model_neg.eval()
        dev_pos = next(model_pos.parameters()).device
        dev_neg = next(model_neg.parameters()).device

        with torch.no_grad():
            s1_pos = model_pos(X_val_upd[(y_val == 1)]).flatten()
            s1_no  = model_pos(X_noise_pos).flatten()
            beta_pos, _ = pick_beta(s1_pos, s1_no)

            s2_neg = model_neg(X_val_upd[(y_val == -1)]).flatten()
            s2_no  = model_neg(X_noise_neg).flatten()
            beta_neg, _ = pick_beta(s2_neg, s2_no)

        cG_grid = [0.1, 0.25, 0.5, 0.75, 1.0]
        best_S, best_tuple, best_m = -float("inf"), None, None
```

```python

        for cG in cG_grid:
            m_try = metrics_from_sheet(
                X_val_upd, y_val, model_pos, model_neg,
                beta_pos=beta_pos, beta_neg=beta_neg
            )
            S_try = m_try["F12"] - cG * m_try["G12"]
            if S_try > best_S:
                best_S, best_tuple, best_m = S_try, (beta_pos, beta_neg, cG), m_try

        m = dict(best_m)
        m["removed_idx"] = i
        m["beta_pos_best"], m["beta_neg_best"], m["cG_best"] = best_tuple
        m["S"] = best_S
        rows.append(m)

        if best_S > best_key:
            best_key = best_S
            best_idx = i

    df = (
        pd.DataFrame(rows)
          .sort_values(["S", "F12", "G12"], ascending=[False, False, True])
          .reset_index(drop=True)
    )
    return best_idx, df

def set_seed(seed: int = 42):
    import random
    import numpy as np
    import torch

    random.seed(seed)

    np.random.seed(seed)

    torch.manual_seed(seed)

    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
```

```python


class ReducerBase:
    def fit(self, X, y=None): ...
    def transform(self, X): ...
    def fit_transform(self, X, y=None):
        self.fit(X, y)
        return self.transform(X)

    @staticmethod
    def _to_numpy(X):
        if torch.is_tensor(X):
            return X.detach().cpu().numpy()
        return np.asarray(X)

    @staticmethod
    def _to_numpy_y(y):
        if y is None:
            return None
        if torch.is_tensor(y):
            return y.detach().cpu().numpy()
        return np.asarray(y)

class MLPReducer(ReducerBase):
    def __init__(self, d_hidden, n_hidden_layers, num_epochs,
       batch_size, n_select, seed=0, output=False, device: str = "cpu
       ",):
        self.d_hidden = d_hidden
        self.n_hidden_layers = n_hidden_layers
        self.num_epochs = num_epochs
        self.batch_size = batch_size
        self.seed = seed
        self.output = output
        self.n_select = n_select

        self.selected_indices_ = None
        self.removed_indices_ = None
        self.history_ = []
        self.device = torch.device(device)


    def _to_torch2d(self, X):
        if isinstance(X, torch.Tensor):
```

```
            assert X.ndim == 2, "X must be 2D"
            return X.to(device=self.device, dtype=torch.float32)
        X = np.asarray(X)
        assert X.ndim == 2, "X must be 2D"
        return torch.from_numpy(X).to(device=self.device, dtype=torch
            .float32)

    def _to_torch1d_int(self, y):
        if isinstance(y, torch.Tensor):
            assert y.ndim == 1, "y must be 1D"
            return y.to(device=self.device, dtype=torch.int64)
        y = np.asarray(y).ravel()
        return torch.from_numpy(y.astype(np.int64)).to(device=self.
            device)


    def fit(self, X, y):
        X_t = self._to_torch2d(X)
        y_t = self._to_torch1d_int(y)

        y_t = y_t.to(device=X_t.device)

        d = X_t.shape[1]
        keep = list(range(d))
        removed = []

        while len(keep) > self.n_select:
            X_cur = X_t[:, keep]

            best_idx, df = choose_coord(
                X=X_cur, y=y_t,
                d_hidden=self.d_hidden,
                n_hidden_layers=self.n_hidden_layers,
                num_epochs=self.num_epochs,
                batch_size=self.batch_size,
                seed=self.seed,
                output=self.output,
            )

            removed_global = keep[best_idx]
            removed.append(removed_global)
            del keep[best_idx]

```

```
            self.history_.append({
                "removed_global": removed_global,
                "n_left": len(keep),
                "df": df,
            })

        self.selected_indices_ = keep
        self.removed_indices_ = removed
        return self

    def transform(self, X):
        assert self.selected_indices_ is not None, "Call fit() before
            transform()."

        if isinstance(X, torch.Tensor):
            return X[:, self.selected_indices_]
        X = np.asarray(X)
        return X[:, self.selected_indices_]

    def fit_transform(self, X, y):
        return self.fit(X, y).transform(X)

    def get_support(self, indices=False):
        assert self.selected_indices_ is not None, "Call fit() first.
            "
        if indices:
            return np.array(self.selected_indices_, dtype=int)

        if self.selected_indices_:
            mask_len = max(self.selected_indices_) + 1
        else:
            mask_len = self.n_select

        mask = np.zeros(mask_len, dtype=bool)
        if self.selected_indices_:
            mask[self.selected_indices_] = True
        return mask
```

Листинг 1. Метод LOFO с использованием унарной классификации

ПРИЛОЖЕНИЕ Б

Ниже приведен код, относящийся к RaSE методу.

```python

def sample_uniform(fmin: torch.Tensor, fmax: torch.Tensor, n: int) -> torch.Tensor:

    fmax = fmax.to(device=fmin.device, dtype=fmin.dtype)


    u = torch.rand(n, fmin.numel(), dtype=fmin.dtype, device=fmin.device)
    return fmin.unsqueeze(0) + (fmax - fmin).unsqueeze(0) * u

def tune_betas_joint(
    model_pos: torch.nn.Module,
    model_neg: torch.nn.Module,
    X_val: torch.Tensor,
    y_val: torch.Tensor,
    fmin: torch.Tensor,
    fmax: torch.Tensor,
    n_grid: int = 19,
    cG_grid: Iterable[float] = (0.25, 0.5, 0.75, 1.0),
) -> Tuple[float, float, float, float]:
    """
    Совместно подбирает (beta_pos, beta_neg) и cG, максимизируя
    S = F12 - cG * G12 на валидации.
    Возвращает: (beta_pos, beta_neg, best_cG, best_S).
    """

    device = next(model_pos.parameters()).device
    Xv = X_val.to(device=device, dtype=torch.float32)
    yv = y_val.to(device=device, dtype=torch.int64)

    model_pos.eval(); model_neg.eval()
    with torch.no_grad():
        s1_all = model_pos(Xv).flatten()
        s2_all = model_neg(Xv).flatten()
        pos = (yv == 1)
        neg = (yv == -1)

        n1 = max(1, int(pos.sum().item()))
        n2 = max(1, int(neg.sum().item()))
        Xno1 = sample_uniform(fmin.to(device), fmax.to(device), n1)
        Xno2 = sample_uniform(fmin.to(device), fmax.to(device), n2)
        s1_no = model_pos(Xno1).flatten()
```

```
        s2_no = model_neg(Xno2).flatten()

        q = torch.linspace(0.05, 0.95, steps=n_grid, device=device)
        pool1 = torch.quantile(torch.cat([s1_all[pos], s1_all[neg],
            s1_no]), q)
        pool2 = torch.quantile(torch.cat([s2_all[neg], s2_all[pos],
            s2_no]), q)

        n1 = max(1, int(pos.sum().item()))
        n2 = max(1, int(neg.sum().item()))

    best_S = -float("inf"); best_bp=0.0; best_bn=0.0; best_cG=1.0

    for bpos_t in pool1:
        bpos = float(bpos_t.item())
        m1 = (s1_all >= bpos)
        for bneg_t in pool2:
            bneg = float(bneg_t.item())
            m2 = (s2_all >= bneg)

            n02_1 = int(((m1 & ~m2) & pos).sum().item())
            n02_2 = int(((m2 & ~m1) & neg).sum().item())
            a1 = n02_1 / n1
            a2 = n02_2 / n2
            F12 = 2*a1*a2 / (a1+a2+1e-12)

            n12_1 = int(((m1 & m2) & pos).sum().item())
            n12_2 = int(((m2 & m1) & neg).sum().item())
            b1 = n12_1 / n1
            b2 = n12_2 / n2
            G12 = 2*b1*b2 / (b1+b2+1e-12)

            for cG in cG_grid:
                S = F12 - float(cG) * G12
                if S > best_S:
                    best_S = S; best_bp=bpos; best_bn=bneg; best_cG=
                        float(cG)

    return best_bp, best_bn, best_cG, best_S


def _project_capped_simplex(p: np.ndarray, L: float, U: float) -> np.
    ndarray:
```

```
    p = np.asarray(p, dtype=float)
    n = p.size
    L = float(L); U = float(U)
    L = min(L, 1.0/n)
    U = max(U, 1.0/n)

    x = np.clip(p, L, U)
    r = 1.0 - x.sum()
    if abs(r) < 1e-12:
        return x / x.sum()

    for _ in range(5):
        if r > 0:
            mask = (x < U - 1e-12)
            if not np.any(mask):
                break
            w = p[mask].copy()
            w[w < 0] = 0.0
            if not np.any(w):
                w = np.ones_like(w)
            w = w / w.sum()
            delta = np.minimum(U - x[mask], r * w / (w.sum() + 1e-12)
                )
            add = delta * (r / (delta.sum() + 1e-12))
            x[mask] += add
            r = 1.0 - x.sum()
        else:
            mask = (x > L + 1e-12)
            if not np.any(mask):
                break
            w = p[mask].copy()
            w[w < 0] = 0.0
            if not np.any(w):
                w = np.ones_like(w)
            w = w / w.sum()
            delta = np.minimum(x[mask] - L, (-r) * w / (w.sum() + 1e
                -12))
            sub = delta * ((-r) / (delta.sum() + 1e-12))
            x[mask] -= sub
            r = 1.0 - x.sum()

        if abs(r) < 1e-12:
            break
```

```python

    x = np.clip(x, L, U)
    x /= x.sum()
    return x

def dynamic_caps(n: int, q: int,
                 alpha_top: float = 0.80,
                 kappa: float = 0.50,
                 u_clip: float = 0.25,
                 eps: float = 1e-6) -> tuple[float, float]:

    assert n >= 1 and 1 <= q <= n, "dynamic_caps: bad n or q"
    U = min(alpha_top / max(q, 1), u_clip)
    U = max(U, 1.0 / n + eps)

    if n > q:
        L_max = max(0.0, (1.0 - q * U) / (n - q))
    else:
        L_max = 0.0

    L = kappa * L_max
    L = max(0.0, min(L, U - eps))
    return L, U


class EnsembleReducer(ReducerBase):
    def __init__(
        self,
        d_hidden,
        n_hidden_layers,
        num_epochs,
        batch_size,
        n_select,
        num_models,
        num_attempts,
        num_coords,
        num_samples,
        seed=0,
        output=False,
        device: str = "cpu",
        use_gpu_shap: bool = False
    ):
        self.d_hidden = d_hidden
```

```
        self.n_hidden_layers = n_hidden_layers
        self.num_epochs = num_epochs
        self.batch_size = batch_size
        self.seed = seed
        self.output = output
        self.n_select = n_select
        self.num_samples = num_samples
        self.num_models = num_models
        self.num_attempts = num_attempts
        self.num_coords = num_coords

        self.list_model_pos = []
        self.list_model_neg = []
        self.list_model_subs = []
        self.list_beta_pos = []
        self.list_beta_neg = []
        self.list_best_cG = []
        self.list_best_S = []
        self.coord_prob = None
        self.history_ = []
        self.rng = np.random.default_rng(self.seed)

        self.selected_indices_ = None

        self.device = torch.device(device)
        self.use_gpu_shap = use_gpu_shap

    def _to_torch2d(self, X):
        if isinstance(X, torch.Tensor):
            assert X.ndim == 2, "X must be 2D"
            return X.to(dtype=torch.float32)
        X = np.asarray(X)
        assert X.ndim == 2, "X must be 2D"
        return torch.from_numpy(X).to(dtype=torch.float32)

    def _to_torch1d_int(self, y):
        if isinstance(y, torch.Tensor):
            assert y.ndim == 1, "y must be 1D"
            return y.to(dtype=torch.int64)
        y = np.asarray(y).ravel()
        return torch.from_numpy(y.astype(np.int64))

    @staticmethod
```

```
    def _shap_mean_pos(expl, X):
        vals = expl.shap_values(X)
        if isinstance(vals, list):
            vals = np.array(vals)
            if vals.ndim == 3:
                vals = vals.mean(axis=0)
        V = np.atleast_2d(np.asarray(vals, float))
        return np.maximum(V, 0.0).mean(axis=0)

    @staticmethod
    def _robust_norm(v: np.ndarray) -> np.ndarray:
        v = np.asarray(v, dtype=float)
        v = np.maximum(v, 0.0)
        if not np.any(np.isfinite(v)):
            return np.full_like(v, 1.0 / max(1, v.size))
        p95 = np.percentile(v[np.isfinite(v)], 95.0)
        if p95 > 0:
            v = np.minimum(v, p95)
        s = v.sum()
        return (v / s) if (np.isfinite(s) and s > 0) else np.
           full_like(v, 1.0 / max(1, v.size))


    def fit(self, X, y):

        X_t = self._to_torch2d(X).to(self.device)
        y_t = self._to_torch1d_int(y).to(self.device)

        n_feat = X_t.shape[1]
        n_samples_total = X_t.shape[0]


        y_np = y_t.detach().cpu().numpy().ravel()
        idx_np = np.arange(n_samples_total)

        train_idx_np, val_idx_np = train_test_split(
            idx_np,
            test_size=0.2,
            random_state=self.seed,
            stratify=y_np
        )

```

```
        train_idx = torch.as_tensor(train_idx_np, dtype=torch.long, device=X_t.device)
        val_idx   = torch.as_tensor(val_idx_np,   dtype=torch.long, device=X_t.device)

        X_tr, y_tr   = X_t[train_idx], y_t[train_idx]
        X_val, y_val = X_t[val_idx],   y_t[val_idx]


        fmin_all, fmax_all = compute_feature_bounds(TensorDataset(X_tr, y_tr))
        fmin_all = fmin_all.to(device=X_tr.device, dtype=X_tr.dtype)
        fmax_all = fmax_all.to(device=X_tr.device, dtype=X_tr.dtype)

        self.coord_prob = np.full(n_feat, 1.0 / n_feat, dtype=float)

        for k in range(self.num_samples):

            for i in range(self.num_models):
                best_model_pos = best_model_neg = None
                best_subs_idx = None
                best_beta_pos = best_beta_neg = None
                best_best_S = -float('inf')
                best_best_cG = None

                for j in range(self.num_attempts):
                    lam = 0.3
                    if self.rng.random() < lam:
                        subs_idx = self.rng.choice(n_feat, size=self.num_coords, replace=False)
                    else:
                        subs_idx = self.rng.choice(
                            n_feat, size=self.num_coords, replace=False, p=self.coord_prob
                        )

                    subs_idx = np.sort(subs_idx)
                    subs_idx_t = torch.as_tensor(subs_idx, dtype=torch.long, device=X_tr.device)

                    X_sub_tr  = X_tr[:, subs_idx_t]
                    X_sub_val = X_val[:, subs_idx_t]

```

```
                    fmin_upd = fmin_all[subs_idx_t]
                    fmax_upd = fmax_all[subs_idx_t]

                    train_ds = TensorDataset(X_sub_tr, y_tr)

                    model_pos, model_neg = model_train(
                        d_hidden=self.d_hidden,
                        n_hidden_layers=self.n_hidden_layers,
                        train_data=train_ds,
                        num_epochs=self.num_epochs,
                        batch_size=self.batch_size,
                        seed=self.seed + 1337 * k + j,
                        output=self.output,
                        device=self.device,
                        feature_min=fmin_upd,
                        feature_max=fmax_upd,
                    )
                    model_pos.eval()
                    model_neg.eval()

                    bpos, bneg, cG_star, S_star = tune_betas_joint(
                        model_pos, model_neg,
                        X_sub_val, y_val,
                        fmin_upd, fmax_upd,
                        n_grid=19,
                        cG_grid=(0.25, 0.5, 0.75, 1.0)
                    )

                    if S_star > best_best_S:
                        best_best_S    = S_star
                        best_subs_idx = subs_idx
                        best_model_pos = model_pos
                        best_model_neg = model_neg
                        best_beta_pos  = bpos
                        best_beta_neg  = bneg
                        best_best_cG   = cG_star

                self.list_model_pos.append(best_model_pos)
                self.list_model_neg.append(best_model_neg)
                self.list_model_subs.append(best_subs_idx)
                self.list_beta_pos.append(best_beta_pos)
                self.list_beta_neg.append(best_beta_neg)
                self.list_best_cG.append(best_best_cG)
```

```
                self.list_best_S.append(best_best_S)

            start = len(self.list_model_pos) - self.num_models
            end   = len(self.list_model_pos)
            S_slice = np.asarray(self.list_best_S[start:end], dtype=
               float)

            keep_top = 2
            keep_top = min(keep_top, max(1, end - start))
            top_local_idx = np.argpartition(-S_slice, kth=min(
               keep_top, S_slice.size) - 1)[:keep_top]
            top_global_idx = (top_local_idx + start).tolist()


            coord_new_prob = np.zeros(n_feat, dtype=float)

            bg_size = min(256, X_tr.shape[0])
            ev_size = min(256, X_val.shape[0])

            ytr_np = y_tr.detach().cpu().numpy().ravel()
            yvl_np = y_val.detach().cpu().numpy().ravel()
            pos_tr = np.flatnonzero(ytr_np ==  1)
            neg_tr = np.flatnonzero(ytr_np == -1)
            pos_vl = np.flatnonzero(yvl_np ==  1)
            neg_vl = np.flatnonzero(yvl_np == -1)

            def _choose_5050(pos_idx, neg_idx, total, seed):
                rng = np.random.default_rng(seed)
                if (len(pos_idx) == 0) and (len(neg_idx) == 0):
                    return np.array([], dtype=int)
                if (len(pos_idx) == 0) or (len(neg_idx) == 0):
                    pool = np.concatenate([pos_idx, neg_idx])
                    total_eff = min(total, len(pool))
                    return rng.choice(pool, size=total_eff, replace=
                       False)

                n_pos = total // 2
                n_neg = total - n_pos
                n_pos = min(n_pos, len(pos_idx))
                n_neg = min(n_neg, len(neg_idx))
                need  = total - (n_pos + n_neg)

                if need > 0:
```

```
                rem_pos = len(pos_idx) - n_pos
                add = min(need, max(0, rem_pos))
                n_pos += add
                need  -= add
            if need > 0:
                rem_neg = len(neg_idx) - n_neg
                add = min(need, max(0, rem_neg))
                n_neg += add
                need  -= add

            chosen_pos = (np.random.default_rng(seed).choice(pos_idx, size=n_pos, replace=False)
                          if n_pos > 0 else np.array([], dtype=int))
            chosen_neg = (np.random.default_rng(seed + 1).choice(neg_idx, size=n_neg, replace=False)
                          if n_neg > 0 else np.array([], dtype=int))
            chosen = np.concatenate([chosen_pos, chosen_neg])
            rng.shuffle(chosen)
            return chosen

        bg_idx_np = _choose_5050(pos_tr, neg_tr, bg_size, seed=self.seed + k)
        ev_idx_np = _choose_5050(pos_vl, neg_vl, ev_size, seed=self.seed + k + 1)

        if bg_idx_np.size == 0:
            bg_idx_np = np.arange(X_tr.shape[0])
        if ev_idx_np.size == 0:
            ev_idx_np = np.arange(X_val.shape[0])

        bg_idx = torch.as_tensor(bg_idx_np, dtype=torch.long, device=X_tr.device)
        ev_idx = torch.as_tensor(ev_idx_np, dtype=torch.long, device=X_val.device)

        for i_top in top_global_idx:
            subs = self.list_model_subs[i_top]
            subs_t = torch.as_tensor(subs, dtype=torch.long, device=X_tr.device)

            X_bg   = X_tr[bg_idx][:, subs_t]
```

```
                X_eval = X_val[ev_idx][:, subs_t]
                y_eval = y_val[ev_idx]

                X_pos = X_eval[(y_eval ==  1)]
                X_neg = X_eval[(y_eval == -1)]
                if X_pos.shape[0] == 0:
                    X_pos = X_eval
                if X_neg.shape[0] == 0:
                    X_neg = X_eval

                if self.use_gpu_shap and self.device.type == "cuda":
                    mp = copy.deepcopy(self.list_model_pos[i_top]).to
                       (self.device).eval()
                    mn = copy.deepcopy(self.list_model_neg[i_top]).to
                       (self.device).eval()

                    X_bg_expl  = X_bg
                    X_pos_expl = X_pos
                    X_neg_expl = X_neg

                else:

                    mp = copy.deepcopy(self.list_model_pos[i_top]).to
                       ("cpu").eval()
                    mn = copy.deepcopy(self.list_model_neg[i_top]).to
                       ("cpu").eval()

                    X_bg_expl  = X_bg.detach().cpu()
                    X_pos_expl = X_pos.detach().cpu()
                    X_neg_expl = X_neg.detach().cpu()


                expl_pos = shap.GradientExplainer(mp, X_bg_expl)
                expl_neg = shap.GradientExplainer(mn, X_bg_expl)

                mu_pos = self._shap_mean_pos(expl_pos, X_pos_expl)
                mu_neg = self._shap_mean_pos(expl_neg, X_neg_expl)

                mu_pos = self._robust_norm(mu_pos)
                mu_neg = self._robust_norm(mu_neg)

                mu_pos = np.squeeze(np.asarray(mu_pos, dtype=float)).
                   reshape(-1)
```

```
                mu_neg = np.squeeze(np.asarray(mu_neg, dtype=float)).reshape(-1)
                assert mu_pos.shape == mu_neg.shape, f"Shape mismatch: {mu_pos.shape} vs {mu_neg.shape}"

                imp_sub = 2.0 * mu_pos * mu_neg / (mu_pos + mu_neg + 1e-12)

                s = imp_sub.sum()
                if s > 0 and np.isfinite(s):
                    imp_sub /= s
                coord_new_prob[subs] += imp_sub

            tot = coord_new_prob.sum()
            if not np.isfinite(tot) or tot <= 0:
                coord_new_prob[:] = 1.0 / n_feat
            else:
                coord_new_prob /= tot

            alpha = 0.35
            coord_smooth = (1.0 - alpha) * self.coord_prob + alpha * coord_new_prob

            L, U = dynamic_caps(n_feat, self.n_select, alpha_top=0.80, kappa=0.50, u_clip=0.25)
            coord_smooth = _project_capped_simplex(coord_smooth, L=L, U=U)

            self.coord_prob = coord_smooth

            if self.output:
                print("pi:", np.round(self.coord_prob, 4))


        return self

    def coord_prob_get(self):
        return self.coord_prob

    def transform(self, X):
        assert self.coord_prob is not None, "Call fit() before transform()."
```

```
        assert hasattr(self, "n_select"), "n_select must be set in __init__()."

        k = min(self.n_select, len(self.coord_prob))
        topk_idx = np.argsort(self.coord_prob)[-k:]
        topk_idx = np.sort(topk_idx)
        self.selected_indices_ = topk_idx

        if isinstance(X, torch.Tensor):
            assert X.ndim == 2, "X must be 2D tensor"
            return X[:, topk_idx]
        else:
            X = np.asarray(X)
            assert X.ndim == 2, "X must be 2D array"
            return X[:, topk_idx]
```

Листинг 2. Метода RaSE с использованием унарной классификации и SHAP

ПРИЛОЖЕНИЕ В

Ниже приведен код относящийся к проведению экспериментов по сравнению методов с существующими.

```

def make_dataset_A(seed=42):
    X, y = make_classification(
        n_samples=3000, n_features=8, n_informative=3, n_redundant=2,
        class_sep=1.2, flip_y=0.02, random_state=seed)
    X[:, 0] = np.sin(X[:, 0]) + 0.1 * X[:, 1]
    X[:, 2] = X[:, 2] ** 2
    y = y.astype(int)
    y = 2*y - 1
    return X.astype(np.float64), y.astype(int)

def make_dataset_B(seed=42):
    X, y = make_classification(
        n_samples=6000, n_features=30, n_informative=5, n_redundant=5,
        class_sep=0.8, flip_y=0.01, weights=[0.75, 0.25], random_state=seed)
    X[:, 10] = np.sin(X[:, 0]) * X[:, 1]
    X[:, 11] = X[:, 2] ** 2
    X[:, 12] = np.exp(-X[:, 3]**2)
    X[:, 20:25] = X[:, :5] + 0.05 * np.random.randn(len(X), 5)
```

```
    y = y.astype(int)
    y = 2*y - 1
    return X.astype(np.float64), y.astype(int)

def make_dataset_A_real(seed: int = 42) -> tuple[np.ndarray, np.
   ndarray]:
    ds = fetch_ucirepo(id=545)
    X_df = ds.data.features
    y_df = ds.data.targets

    X = X_df.to_numpy(dtype=np.float64)
    y_raw = np.asarray(y_df.squeeze())

    classes = np.unique(y_raw)

    pos_label = "Osmancik" if "Osmancik" in classes else classes[-1]
    y = np.where(y_raw == pos_label, 1, -1).astype(int)

    return X.astype(np.float64), y.astype(int)

def make_dataset_B_real(seed: int = 42) -> tuple[np.ndarray, np.
   ndarray]:
    ds = fetch_ucirepo(id=327)
    X = ds.data.features.to_numpy(dtype=np.float64)
    y_raw = np.asarray(ds.data.targets.squeeze())

    y_num = y_raw.astype(int)

    y = np.where(y_num > 0, 1, -1).astype(int)

    return X.astype(np.float64), y.astype(int)

class PCAReducer(ReducerBase):
    def __init__(self, n_components):
        self.pca = PCA(n_components=n_components, random_state=42)

    def fit(self, X, y=None):
        Xn = self._to_numpy(X)
        self.pca.fit(Xn)
        return self

    def transform(self, X):
        Xn = self._to_numpy(X)
```

```
        return self.pca.transform(Xn)

class PLSReducer(ReducerBase):
    def __init__(self, n_components):
        self.pls = PLSRegression(n_components=n_components, scale=
           False)

    def fit(self, X, y):
        Xn = self._to_numpy(X)
        yn = self._to_numpy_y(y).ravel().astype(float)
        self.pls.fit(Xn, yn)
        return self

    def transform(self, X):
        Xn = self._to_numpy(X)
        return self.pls.transform(Xn)


class UMAPReducer(ReducerBase):
    def __init__(self, n_components):
        self.umap = UMAP(
            n_components=n_components,
            random_state=42,
            n_neighbors=15,
            min_dist=0.1,
            target_metric='categorical'
        )

    def fit(self, X, y):
        Xn = self._to_numpy(X)
        yn = self._to_numpy_y(y)
        self.umap.fit(Xn, yn)
        return self

    def transform(self, X):
        Xn = self._to_numpy(X)
        return self.umap.transform(Xn)

class HSICSelector(ReducerBase):
    def __init__(self, n_select):
        self.n_select = int(n_select)
        self.model = HSICLasso()
        self.mask_ = None
```

```python

    def fit(self, X, y):
        Xn = self._to_numpy(X)
        yn = self._to_numpy_y(y).ravel().astype(int)

        self.model.input(Xn, yn)
        self.model.classification(self.n_select)

        idx = np.array(self.model.get_index(), dtype=int)
        self.mask_ = np.zeros(Xn.shape[1], dtype=bool)
        self.mask_[idx] = True
        return self

    def transform(self, X):
        Xn = self._to_numpy(X)
        return Xn[:, self.mask_]

def get_models():
    return {
        "MLP": MLPClassifier(hidden_layer_sizes=(64,32), activation='relu',
                             alpha=1e-4, batch_size=256, learning_rate_init=1e-3,
                             max_iter=200, early_stopping=True, n_iter_no_change=10, random_state=42),
        "HGBT": HistGradientBoostingClassifier(random_state=42)
    }


def evaluate_reduction(
    X: np.ndarray,
    y: np.ndarray,
    reducer_name: str,
    reducer_ctor: Any,
    q: int,
    dataset_name: str,
    n_splits: int = 5,
    reducer_kwargs: Dict[str, Any] = None,
    compute_unary_on_reduced: bool = True,
    unary_params: Dict[str, Any] = None,
) -> Tuple[List[Dict[str, Any]], List[Dict[str, Any]]]:

```

```python

    reducer_kwargs = reducer_kwargs or {}
    unary_params = unary_params or dict(
        d_hidden=48,
        n_hidden_layers=2,
        num_epochs=60,
        batch_size=256,
        beta_grid=31,
        cG_grid=(0.10, 0.20, 0.25, 0.33, 0.50, 0.67, 0.75, 1.00, 1.25),

    )

    y_orig = np.asarray(y).ravel()
    if set(np.unique(y_orig)) == {-1, 1}:
        y_bin = (y_orig == 1).astype(int)
    else:
        y_bin = y_orig.astype(int)

    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    rows_cls: List[Dict[str, Any]] = []
    rows_unary: List[Dict[str, Any]] = []

    for fold, (tr, te) in enumerate(skf.split(X, y_bin), 1):
        Xtr, Xte = X[tr], X[te]
        ytr_bin, yte_bin = y_bin[tr], y_bin[te]
        ytr_orig, yte_orig = y_orig[tr], y_orig[te]


        scaler = StandardScaler()
        Xtr_s = scaler.fit_transform(Xtr)
        Xte_s = scaler.transform(Xte)


        base_scores = {}
        models = get_models()
        for mname, base_model in models.items():
            m0 = base_model.__class__(**base_model.get_params())
            t0 = perf_counter()
            m0.fit(Xtr_s, ytr_bin)
            t0 = perf_counter() - t0
```

```
            p0 = m0.predict_proba(Xte_s)[:, 1] if hasattr(m0, "predict_proba") else m0.decision_function(Xte_s)
            auc0 = roc_auc_score(yte_bin, p0)
            ap0 = average_precision_score(yte_bin, p0)
            base_scores[mname] = (auc0, ap0, t0)


        red = reducer_ctor(q=q, **reducer_kwargs)
        tR = perf_counter()
        Ztr = red.fit_transform(Xtr_s, ytr_orig)
        Zte = red.transform(Xte_s)
        tR = perf_counter() - tR


        for mname, base_model in models.items():
            auc0, ap0, t0 = base_scores[mname]
            m1 = base_model.__class__(**base_model.get_params())
            t1 = perf_counter()
            m1.fit(Ztr, ytr_bin)
            t1 = perf_counter() - t1
            p1 = m1.predict_proba(Zte)[:, 1] if hasattr(m1, "predict_proba") else m1.decision_function(Zte)
            auc1 = roc_auc_score(yte_bin, p1)
            ap1 = average_precision_score(yte_bin, p1)

            rows_cls.append({
                "dataset": dataset_name, "method": reducer_name, "q": q, "model": mname, "fold": fold,
                "AUC": auc1, "PR_AUC": ap1, "delta_AUC": auc1 - auc0, "delta_PR_AUC": ap1 - ap0,
                "t_reducer": tR, "t_model": t1, "t_model_base": t0
            })


        if compute_unary_on_reduced:
            dev_param = unary_params.get("device", None)
            if dev_param is not None:
                unary_device_preferred = torch.device(dev_param)
            else:
                if torch.cuda.is_available():
                    unary_device_preferred = torch.device("cuda")
                elif torch.backends.mps.is_available():
                    unary_device_preferred = torch.device("mps")
```

```
            else:
                unary_device_preferred = torch.device("cpu")

        ytr_pm = np.where(ytr_orig == 1, 1, -1)
        yte_pm = np.where(yte_orig == 1, 1, -1)

        sss = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
        (fit_idx, val_idx), = sss.split(Ztr, (ytr_pm == 1).astype(int))
        Ztr_fit, Ztr_val = Ztr[fit_idx], Ztr[val_idx]
        y_fit_pm, y_val_pm = ytr_pm[fit_idx], ytr_pm[val_idx]


        X_fit_t = torch.as_tensor(Ztr_fit, dtype=torch.float32, device=unary_device_preferred)
        y_fit_t = torch.as_tensor(y_fit_pm, dtype=torch.int64, device=unary_device_preferred)
        train_ds_fit = TensorDataset(X_fit_t, y_fit_t)

        model_pos, model_neg = model_train(
            d_hidden=unary_params.get("d_hidden", 32),
            n_hidden_layers=unary_params.get("n_hidden_layers", 1),
            train_data=train_ds_fit,
            num_epochs=unary_params.get("num_epochs", 50),
            batch_size=unary_params.get("batch_size", 256),
            seed=unary_params.get("seed", 0),
            output=False,
        )
        model_pos.eval()
        model_neg.eval()

        unary_device = next(model_pos.parameters()).device

        X_val_t = torch.as_tensor(Ztr_val, dtype=torch.float32, device=unary_device)
        y_val_t = torch.as_tensor(y_val_pm, dtype=torch.int64, device=unary_device)

        fmin, fmax = compute_feature_bounds(TensorDataset(X_fit_t, y_fit_t))
        fmin = fmin.to(unary_device)
```

```
            fmax = fmax.to(unary_device)

            beta_pos, beta_neg, cG_star, S_val = tune_betas_joint(
                model_pos, model_neg,
                X_val_t, y_val_t,
                fmin, fmax,
                n_grid=unary_params.get("beta_grid", 19),
                cG_grid=unary_params.get("cG_grid", (0.25, 0.5, 0.75, 1.0))
            )


            X_te_t = torch.as_tensor(Zte, dtype=torch.float32, device=unary_device)
            yte_pm_t = torch.as_tensor(yte_pm, dtype=torch.int64, device=unary_device)

            m_un = metrics_from_sheet(
                X_te_t, yte_pm_t,
                model_pos, model_neg,
                beta_pos=float(beta_pos), beta_neg=float(beta_neg)
            )


            with torch.no_grad():
                s1_te = model_pos(X_te_t).flatten()
                s2_te = model_neg(X_te_t).flatten()

            pos_hit = (s1_te >= float(beta_pos)) & (s2_te <  float(beta_neg))
            neg_hit = (s2_te >= float(beta_neg)) & (s1_te <  float(beta_pos))
            coverage = float((pos_hit | neg_hit).float().mean().item())

            S_test = float(m_un["F12"] - cG_star * m_un["G12"])

            rows_unary.append({
                "dataset": dataset_name, "method": reducer_name, "q": q, "model": "UnaryMLP", "fold": fold,
                "F12": float(m_un["F12"]), "G12": float(m_un["G12"]),
                "a1": float(m_un["a1"]), "a2": float(m_un["a2"]),
                "b1": float(m_un["b1"]), "b2": float(m_un["b2"]),
```

```
                "n1": int(m_un["n1"]), "n2": int(m_un["n2"]),
                "coverage": coverage,
                "beta_pos": float(beta_pos), "beta_neg": float(beta_neg),
                "cG": float(cG_star), "S_val": float(S_val), "S_test": S_test,
                "t_reducer": tR, "t_model": 0.0,
                "unary_device": str(unary_device),
            })

    return rows_cls, rows_unary

def run_all_A(
    include_unary: bool = False,
    unary_params: dict | None = None,
) -> Tuple[pd.DataFrame, pd.DataFrame]:
    configs = [
        ("DS-A", make_dataset_A, [2, 3, 5]),
        # ("DS-B", make_dataset_B, [5, 10, 15]),
    ]

    methods = [
        ("PCA",         lambda q, **_: PCAReducer(q)),
        ("PLS",         lambda q, **_: PLSReducer(q)),
        ("UMAP_sup",    lambda q, **_: UMAPReducer(q)),
        ("HSIC_Lasso", lambda q, **_: HSICSelector(q)),
        ("MLP_unar",    lambda q, **kw: MLPReducer(
            d_hidden=64,
            n_hidden_layers=2,
            num_epochs=40,
            batch_size=256,
            n_select=q,
            seed=0,
            output=False,
            **kw,
        )),
    ]


    if torch.cuda.is_available():
        DEVICE = torch.device("cuda")
    elif torch.backends.mps.is_available():
        DEVICE = torch.device("mps")
```

```
    else:
        DEVICE = torch.device("cpu")

    print("Using device:", DEVICE)
    if DEVICE.type == "cuda":
        print("CUDA name:", torch.cuda.get_device_name(0))

    base_unary_params = dict(
        d_hidden=32,
        n_hidden_layers=1,
        num_epochs=50,
        batch_size=256,
    )
    if unary_params is not None:
        base_unary_params.update(unary_params)
    base_unary_params["device"] = str(DEVICE)

    rows_cls_all: List[Dict[str, Any]] = []

    for dname, maker, qgrid in configs:
        X, y = maker()
        for name, ctor in methods:
            for q in qgrid:
                if name == "MLP_unar":
                    reducer_kwargs = {
                        "device": str(DEVICE),
                    }
                else:
                    reducer_kwargs = {}

                cls_rows, _un_rows = evaluate_reduction(
                    X, y,
                    reducer_name=name,
                    reducer_ctor=ctor,
                    q=q,
                    dataset_name=dname,
                    reducer_kwargs=reducer_kwargs,
                    compute_unary_on_reduced=include_unary,
                    unary_params=base_unary_params,
                )
                rows_cls_all.extend(cls_rows)


```

```
    df = pd.DataFrame(rows_cls_all)

    for col in ["AUC", "PR_AUC", "delta_AUC", "delta_PR_AUC",
                "t_reducer", "t_model", "t_model_base"]:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    agg = (
        df.groupby(["dataset", "method", "q", "model"], dropna=False)
          .agg(
              AUC_mean=("AUC", "mean"),
              PR_AUC_mean=("PR_AUC", "mean"),
              dAUC_mean=("delta_AUC", "mean"),
              dPR_mean=("delta_PR_AUC", "mean"),
              t_red_med=("t_reducer", "median"),
              t_mod_med=("t_model", "median"),
          )
          .reset_index()
          .sort_values(["dataset", "model", "dAUC_mean"], ascending=[
             True, True, False])
    )

    return df, agg



def run_all_B(
    include_unary: bool = True,
    unary_params: dict | None = None,
) -> Tuple[pd.DataFrame, pd.DataFrame]:
    configs = [
        ("DS-B", make_dataset_B, [5, 10, 15]),
    ]

    methods = [
        ("PCA",         lambda q, **_: PCAReducer(q)),
        ("PLS",         lambda q, **_: PLSReducer(q)),
        ("UMAP_sup",    lambda q, **_: UMAPReducer(q)),
        ("HSIC_Lasso", lambda q, **_: HSICSelector(q)),
        ("Ensembleunar", lambda q, **kw: EnsembleReducer(
            d_hidden=32, n_hidden_layers=2,
            num_epochs=8, batch_size=1024,
            n_select=q,
```

```
            num_models=4,
            num_attempts=20,
            num_coords=4,
            num_samples=10,
            seed=42, output=True,
            **kw,
        )),
    ]

    if torch.cuda.is_available():
        DEVICE = torch.device("cuda")
    elif torch.backends.mps.is_available():
        DEVICE = torch.device("mps")
    else:
        DEVICE = torch.device("cpu")

    print("Using device:", DEVICE)
    if DEVICE.type == "cuda":
        print("CUDA name:", torch.cuda.get_device_name(0))

    base_unary_params = dict(
        d_hidden=32,
        n_hidden_layers=1,
        num_epochs=50,
        batch_size=256,
    )
    if unary_params is not None:
        base_unary_params.update(unary_params)
    base_unary_params["device"] = str(DEVICE)

    rows_cls_all: List[Dict[str, Any]] = []

    for dname, maker, qgrid in configs:
        X, y = maker()
        for name, ctor in methods:
            for q in qgrid:

                if name == "Ensembleunar":
                    reducer_kwargs = {
                        "device": str(DEVICE),
                        "use_gpu_shap": True,
                    }

```

```
                else:
                    reducer_kwargs = {}

                cls_rows, _un_rows = evaluate_reduction(
                    X, y,
                    reducer_name=name,
                    reducer_ctor=ctor,
                    q=q,
                    dataset_name=dname,
                    reducer_kwargs=reducer_kwargs,
                    compute_unary_on_reduced=include_unary,
                    unary_params=base_unary_params,
                )
                rows_cls_all.extend(cls_rows)


    df = pd.DataFrame(rows_cls_all)

    for col in ["AUC", "PR_AUC", "delta_AUC", "delta_PR_AUC",
                "t_reducer", "t_model", "t_model_base"]:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    agg = (
        df.groupby(["dataset", "method", "q", "model"], dropna=False)
          .agg(
              AUC_mean=("AUC", "mean"),
              PR_AUC_mean=("PR_AUC", "mean"),
              dAUC_mean=("delta_AUC", "mean"),
              dPR_mean=("delta_PR_AUC", "mean"),
              t_red_med=("t_reducer", "median"),
              t_mod_med=("t_model", "median"),
          )
          .reset_index()
          .sort_values(["dataset", "model", "dAUC_mean"], ascending=[
             True, True, False])
    )

    results = {
        "df": df,
        "agg": agg,
    }
    with open("results_B", "wb") as f:
```

```
        pickle.dump(results, f)

    return df, agg

def run_all_A_real(
    include_unary: bool = False,
    unary_params: dict | None = None,
) -> Tuple[pd.DataFrame, pd.DataFrame]:

    configs = [
        ("DS-A-real", make_dataset_A_real, [2, 3, 5]),
    ]

    methods = [
        ("PCA",        lambda q, **_: PCAReducer(q)),
        ("PLS",        lambda q, **_: PLSReducer(q)),
        ("UMAP_sup",   lambda q, **_: UMAPReducer(q)),
        ("HSIC_Lasso", lambda q, **_: HSICSelector(q)),
        ("MLP_unar",   lambda q, **kw: MLPReducer(
            d_hidden=64,
            n_hidden_layers=2,
            num_epochs=40,
            batch_size=256,
            n_select=q,
            seed=0,
            output=False,
            **kw,
        )),
    ]

    if torch.cuda.is_available():
        DEVICE = torch.device("cuda")
    elif torch.backends.mps.is_available():
        DEVICE = torch.device("mps")
    else:
        DEVICE = torch.device("cpu")

    print("Using device:", DEVICE)
    if DEVICE.type == "cuda":
        print("CUDA name:", torch.cuda.get_device_name(0))

    base_unary_params = dict(
        d_hidden=32,
```

```
        n_hidden_layers=1,
        num_epochs=50,
        batch_size=256,
    )
    if unary_params is not None:
        base_unary_params.update(unary_params)
    base_unary_params["device"] = str(DEVICE)

    rows_cls_all: List[Dict[str, Any]] = []

    for dname, maker, qgrid in configs:
        X, y = maker()
        for name, ctor in methods:
            for q in qgrid:
                if name == "MLP_unar":
                    reducer_kwargs = {"device": str(DEVICE)}
                else:
                    reducer_kwargs = {}

                cls_rows, _un_rows = evaluate_reduction(
                    X, y,
                    reducer_name=name,
                    reducer_ctor=ctor,
                    q=q,
                    dataset_name=dname,
                    reducer_kwargs=reducer_kwargs,
                    compute_unary_on_reduced=include_unary,
                    unary_params=base_unary_params,
                )
                rows_cls_all.extend(cls_rows)

    df = pd.DataFrame(rows_cls_all)

    for col in ["AUC", "PR_AUC", "delta_AUC", "delta_PR_AUC",
                "t_reducer", "t_model", "t_model_base"]:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    agg = (
        df.groupby(["dataset", "method", "q", "model"], dropna=False)
          .agg(
              AUC_mean=("AUC", "mean"),
              PR_AUC_mean=("PR_AUC", "mean"),
```

```
                dAUC_mean=("delta_AUC", "mean"),
                dPR_mean=("delta_PR_AUC", "mean"),
                t_red_med=("t_reducer", "median"),
                t_mod_med=("t_model", "median"),
            )
            .reset_index()
            .sort_values(["dataset", "model", "dAUC_mean"], ascending=[
                True, True, False])
    )

    return df, agg

def run_all_B_real(
    include_unary: bool = True,
    unary_params: dict | None = None,
) -> Tuple[pd.DataFrame, pd.DataFrame]:

    configs = [
        ("DS-B-real", make_dataset_B_real, [5, 10, 15]),
    ]

    methods = [
        ("PCA",         lambda q, **_: PCAReducer(q)),
        ("PLS",         lambda q, **_: PLSReducer(q)),
        ("UMAP_sup",    lambda q, **_: UMAPReducer(q)),
        ("HSIC_Lasso", lambda q, **_: HSICSelector(q)),
        ("Ensembleunar", lambda q, **kw: EnsembleReducer(
            d_hidden=32,
            n_hidden_layers=2,
            num_epochs=8,
            batch_size=1024,
            n_select=q,
            num_models=4,
            num_attempts=20,
            num_coords=4,
            num_samples=10,
            seed=42,
            output=True,
            **kw,
        )),
    ]

    if torch.cuda.is_available():
```

```
        DEVICE = torch.device("cuda")
    elif torch.backends.mps.is_available():
        DEVICE = torch.device("mps")
    else:
        DEVICE = torch.device("cpu")

    print("Using device:", DEVICE)
    if DEVICE.type == "cuda":
        print("CUDA name:", torch.cuda.get_device_name(0))

    base_unary_params = dict(
        d_hidden=32,
        n_hidden_layers=1,
        num_epochs=50,
        batch_size=256,
    )
    if unary_params is not None:
        base_unary_params.update(unary_params)
    base_unary_params["device"] = str(DEVICE)

    rows_cls_all: List[Dict[str, Any]] = []

    for dname, maker, qgrid in configs:
        X, y = maker()
        for name, ctor in methods:
            for q in qgrid:
                if name == "Ensembleunar":
                    reducer_kwargs = {
                        "device": str(DEVICE),
                        "use_gpu_shap": True,
                    }
                else:
                    reducer_kwargs = {}

                cls_rows, _un_rows = evaluate_reduction(
                    X, y,
                    reducer_name=name,
                    reducer_ctor=ctor,
                    q=q,
                    dataset_name=dname,
                    reducer_kwargs=reducer_kwargs,
                    compute_unary_on_reduced=include_unary,
                    unary_params=base_unary_params,
```

```
                )
                rows_cls_all.extend(cls_rows)

    df = pd.DataFrame(rows_cls_all)

    for col in [
        "AUC", "PR_AUC", "delta_AUC", "delta_PR_AUC",
        "t_reducer", "t_model", "t_model_base"
    ]:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    agg = (
        df.groupby(["dataset", "method", "q", "model"], dropna=False)
          .agg(
              AUC_mean=("AUC", "mean"),
              PR_AUC_mean=("PR_AUC", "mean"),
              dAUC_mean=("delta_AUC", "mean"),
              dPR_mean=("delta_PR_AUC", "mean"),
              t_red_med=("t_reducer", "median"),
              t_mod_med=("t_model", "median"),
          )
          .reset_index()
          .sort_values(
              ["dataset", "model", "dAUC_mean"],
              ascending=[True, True, False],
          )
    )

    results = {
        "df": df,
        "agg": agg,
    }
    with open("results_B_real", "wb") as f:
        pickle.dump(results, f)

    return df, agg
```

Листинг 3. Проводимые эксперименты